MACHINE LEARNING · WEEK 2 · LECTURE NOTES

Lecture 2: From Risk Minimisation to Linear Regression

Mathematical revision · empirical risk minimisation · baselines · least squares · gradient descent · ridge and lasso regularisation

Item	Details
Programme	
Syllabus coverage	Week 2: mathematical revision, empirical risk minimisation · Week 2: linear regression, least squares, gradient descent, ridge and lasso
Lecturer	Behnam Kiani · Kiani-bme@hotmail.com · office hours by appointment
Companion materials	Slides (68, with speaker notes, build animations and 3 embedded videos) · Lab 2 handout · starter and solution notebooks
Data	Palmer penguins (Gorman, Williams & Fraser 2014; CC0) · scikit-learn diabetes data (Efron et al. 2004)
Core reading	ISLP Ch. 3 & §6.2 · Géron Ch. 4 · Deisenroth, Faisal & Ong, Mathematics for ML, Ch. 2, 5, 9

How to use these notes

The notes follow the slides. Each section gives the **key points**, the **equations** and **code** shown on the slide, and a full **explanation**. Appendix A collects every derivation step by step; Appendix B lists the key numbers; Appendix C contains self-check questions with answers.

The one-sentence summary

Choose a hypothesis class and a loss, minimise the average training loss, by formula or by gradient descent, and control complexity with regularisation.

Introduction

Welcome to Lecture 2. Last week we saw the big picture: machine learning means learning a model from data and answers, a model is a function with adjustable parameters, and learning means adjusting those parameters to reduce error. Today we make every one of those words precise.

We follow our new syllabus. From Week 1 we still owe you two topics: a mathematical revision, the linear algebra, calculus and probability you need for the rest of the course, and empirical risk minimisation, the single principle behind almost every learning algorithm. Then we cover all of Week 2: linear regression, the method of least squares, gradient descent, and ridge and lasso regularisation.

The picture on the right summarises the lecture: real data on 342 penguins, a straight line fitted by minimising squared error, and the update rule of gradient descent, which we will see in action. By the end you will be able to derive, implement from scratch and regularise a linear regression model, which is exactly what you will do in today's lab.

Where this lecture fits in the new syllabus

Slide 2

Key points

- Week 1: paradigms ✓ & problem formulation ✓ (Lecture 1); maths revision & ERM → today
- Week 2: linear regression, least squares, gradient descent, ridge & lasso → today
- Lab: NumPy exercises + baseline (Week 1) and regression & GD from scratch (Week 2)
- Next: logistic regression, maximum likelihood, Naive Bayes (Week 3)

Explanation

Here is how today's lecture maps onto the new 16-week syllabus. Week 1 had four lecture topics. Learning paradigms, supervised, unsupervised and reinforcement learning, and problem formulation, through the Task–Experience–Performance framework, were covered in Lecture 1. Mathematical revision and empirical risk minimisation were not yet covered, so they form the first half of today.

Week 2's topics, linear regression, least squares, gradient descent, and ridge and lasso, form the second half. The lab follows the same logic: NumPy exercises and a baseline model from the Week 1 lab, plus implementing regression and gradient descent from scratch from the Week 2 lab. Dataset exploration was already practised in the previous labs with pandas.

Next week we continue with logistic regression, maximum likelihood and Naive Bayes; the probability revision today prepares you for that. Notice how the syllabus builds: today's two ideas, minimise an empirical loss, and use gradients to do it, reappear in every remaining model, up to neural networks and transformers in Weeks 11 to 13.

From Lecture 1 to Lecture 2: making it precise

Slide 3

Key points

- Model with knobs → hypothesis h(x; w) from a class H
- Learning = reducing error → minimise empirical risk
- Small adjustments → gradient descent $w \leftarrow w - \eta \nabla J(w)$
- Care about new data → true risk vs empirical risk

Explanation

Let's connect to Lecture 1. On the left are four informal statements we made last week. On the right is the precise version you will be able to write by the end of today.

“A model is a function with knobs” becomes a hypothesis h from a hypothesis class, with parameters w, for example the linear function w-transpose-x. “Learning means reducing error” becomes minimising the empirical risk: the average loss over the training examples. “Adjust the knobs a little at a time” becomes the gradient descent update: w minus eta times the gradient. And “we care about new data” becomes the distinction between empirical risk, which we can compute, and true risk, which we actually care about but cannot compute.

This shift from words to mathematics is not decoration. It's what lets us derive algorithms, prove when they work, implement them correctly from scratch, and reason about failure, which is the level the rest of this course works at.

Today's learning objectives

Slide 4

Key points

- Use ML notation, linear algebra, gradients, convexity, expectation
- State ERM; formulate a problem as (X, Y, H, ℓ)
- Derive and compute a baseline; mean minimises squared error
- Derive the normal equations; geometric interpretation
- Implement gradient descent; learning rate, scaling, SGD
- Explain ridge and lasso; coefficient shrinkage and sparsity; choosing λ

Explanation

By the end of today you should be able to do six things. One: use the notation and the core linear algebra, calculus and probability of machine learning, shapes of matrices, gradients, convexity, expectation. Two: state the empirical risk minimisation principle and formulate a supervised problem in its terms: input space, output space, hypothesis class and loss. Three: derive and compute a baseline, and explain why the mean minimises squared error. Four: derive the least-squares solution of linear regression, the normal equations, and interpret it geometrically. Five: implement

gradient descent, choose a learning rate, and explain feature scaling and stochastic variants. And six: explain ridge and lasso regularisation, their effect on coefficients and how to choose the penalty.

These map onto the lab: objectives one, three, four, five and six are exactly what you will implement this afternoon, with NumPy and without scikit-learn doing the work for you.

Agenda

Slide 5

Key points

- 01 Maths toolkit
- 02 Learning as risk minimisation
- 03 Baselines
- 04 Linear regression & least squares
- 05 Gradient descent
- 06 Regularisation: ridge & lasso
- 07 Lab & next steps

Explanation

Seven parts today. Part one is a compact mathematical toolkit. Part two formalises learning as risk minimisation. Part three uses that principle to derive baselines, the simplest models of all. Part four is linear regression and least squares, including the derivation of the normal equations. Part five is gradient descent, with two animations. Part six is regularisation: ridge and lasso, with a geometric animation. Part seven previews the lab.

The lecture is mathematical, but every equation is followed by a concrete number from real data and a few lines of NumPy, so you can always check your understanding. If an equation looks unfamiliar, don't panic, the lecture notes contain every step of every derivation.

Part 1, Maths toolkit

The linear algebra, calculus and probability you need

This part is a compact revision. You have met most of it in your mathematics courses; here we use machine-learning notation and focus only on what we need today and in the coming weeks: vectors and matrices and their shapes, products and norms, solving linear systems, derivatives and gradients, a little matrix calculus, convexity, and expectation and variance.

Notation: scalars, vectors, matrices

Slide 7

Key points

- Scalar: x, $\hat{y}$, η · vector (bold): x, w ∈ $\mathbb{R}^d$ · matrix: X
- Data matrix X ∈ $\mathbb{R}^{n \times d}$: n rows (examples), d columns (features)
- Row i = x_i^T; targets y ∈ $\mathbb{R}^n$
- Penguins: n = 342, d = 1 (or 2 with a column of ones)
- Habit: always check shapes (.shape)

Equations

$$\mathbf{x} = (x_1, x_2, \ldots, x_d)^\mathsf{T} \in \mathbb{R}^d$$

Explanation

Let's fix notation for the whole course. Plain italic letters are scalars, single numbers, such as a prediction $\hat{y}$ or a learning rate η. Bold lower-case letters are column vectors, such as one example x with d features, or the weight vector w. Capital letters are matrices, most importantly the data matrix X.

The data matrix has one row per example and one column per feature, so it has shape n by d: n examples, d features. Row i is the transpose of example x_i. Column j holds feature j for every example. The targets are collected in a vector y of length n. In the penguins example from last week, n is 342 and, with only flipper length as a feature, d is 1, or 2 if we include a column of ones for the intercept, as we'll see.

Keeping track of shapes is the single most useful habit in machine-learning mathematics and code. Most bugs in from-scratch implementations are shape errors, and NumPy will tell you about them if you print .shape often.

Matrix–vector products: all predictions at once

Slide 8

Key points

- X (n×d) · w (d) = $\hat{y}$ (n): one prediction per example

- Inner dimensions must match; result has outer dimensions
- NumPy: X @ w (matrix product), not X × w (element-wise)
- ŷ = X @ w is the prediction step of linear models and NN layers

Equations

$$\hat{\mathbf{y}} = X\mathbf{w}, \quad X \in \mathbb{R}^{n \times d},\ \mathbf{w} \in \mathbb{R}^{d},\ \hat{\mathbf{y}} \in \mathbb{R}^{n}$$

Code

```
y_hat = X @ w       # (n, d) @ (d,) -> (n,)
X * w               # element-wise: (n, d), a classic bug
```

Explanation

The most important operation today is the matrix–vector product. Multiplying the data matrix X, of shape n by d, by the weight vector w, of length d, gives a vector of length n: one prediction for every example, all at once. Each entry is the dot product of one row of X with w.

The shape rule: the inner dimensions must match, n-by-d times d-by-1, and the result has the outer dimensions, n by 1. If they don't match, NumPy raises an error saying the shapes are not aligned; that message is your friend.

In NumPy you write X at w, the at-sign is the matrix multiplication operator. The star operator is different: it multiplies element by element and, with broadcasting, can silently give a result of the wrong shape. This is a classic bug: if you write X star w where you meant X at w, you may get an n-by-d matrix instead of n predictions, and everything after that is wrong.

This one line, y-hat equals X at w, is the whole prediction step of linear regression, and later of every layer of a neural network.

Dot products and norms

Slide 9

Key points

- $w^T x = \Sigma\, w_j x_j$: weighted sum of features (linear score)
- Dot product 0 ⇔ vectors orthogonal
- $\|w\|_2^2 = \Sigma\, w_j^2$ (squared length) · $\|w\|_1 = \Sigma\, |w_j|$
- (3, −4): $\|\cdot\|_2 = 5$, $\|\cdot\|_1 = 7$
- $\|Xw - y\|_2^2$ = total squared error; norms of w = ridge/lasso penalties

Equations

$$\mathbf{w}^\top \mathbf{x} = \sum_{j=1}^{d} w_j x_j$$

$$\|\mathbf{w}\|_2^2 = \sum_j w_j^2 \qquad \|\mathbf{w}\|_1 = \sum_j |w_j|$$

Explanation

The dot product of two vectors multiplies them entry by entry and sums the results. In machine learning it appears everywhere: w-transpose-x is a weighted sum of the features, the score of a linear model. Geometrically, the dot product is large when two vectors point in a similar direction and zero when they are perpendicular, or orthogonal. We will use orthogonality to understand least squares geometrically.

A norm measures the size of a vector. The L2 norm, or Euclidean norm, is the ordinary length: the square root of the sum of squares. We usually use its square, the sum of squared entries. The L1 norm is the sum of absolute values. For the vector (3, −4), the L2 norm is 5, the classic 3-4-5 triangle, and the L1 norm is 7.

Norms play two roles today. First, the squared L2 norm of the residual vector, Xw minus y, is the total squared error. Second, the norms of the weight vector are exactly the penalties in ridge regression, which uses L2, and lasso, which uses L1. Remember these two norms; they come back in Part six.

Transpose, inverse and solving linear systems

Slide 10

Key points

- Transpose: $(n \times d)^T = d \times n$; X^TX is d×d and symmetric
- Inverse exists only for full rank (no collinear columns, $n \geq d$)
- Collinear features or $d > n \rightarrow$ no unique least-squares solution
- Solve Aw = b with np.linalg.solve, don't compute inv(A)
- np.linalg.lstsq solves least squares directly

Code

```
w = np.linalg.solve(X.T @ X, X.T @ y)          # preferred
w, *_ = np.linalg.lstsq(X, y, rcond=None)      # stable, handles singular cases
```

Explanation

Three more matrix ideas. The transpose flips a matrix across its diagonal, so an n-by-d matrix becomes d-by-n. A product we'll meet constantly is X-transpose-X: it has shape d by d, small,

because d is usually much less than n, and it's symmetric. Its entries measure how features vary together.

The inverse of a square matrix A undoes it: A-inverse times A is the identity. It exists only when A has full rank, no column is a combination of the others. For X-transpose-X this fails when features are perfectly collinear, for example height in centimetres and height in inches, or when there are more features than examples. Then least squares has no unique solution, one motivation for ridge regression later.

A practical rule: almost never compute an inverse explicitly. To solve A w = b, call np.linalg.solve(A, b). It is faster and numerically more accurate than np.linalg.inv(A) at b. Even better for least squares, np.linalg.lstsq solves the problem directly and copes with rank deficiency.

Derivatives: slopes tell us which way is down

Slide 11

Key points

- Derivative = slope of the tangent = rate of change
- f(w) = (w − 3)² + 1 → f′(w) = 2(w − 3)
- Slope < 0 → increase w to go down; slope > 0 → decrease w
- Minimum where f′(w) = 0 (here w = 3)
- Move against the slope = gradient descent; set to zero = closed form

Equations

$$f(w) = (w-3)^2 \quad \Rightarrow \quad f'(w) = 2(w-3)$$

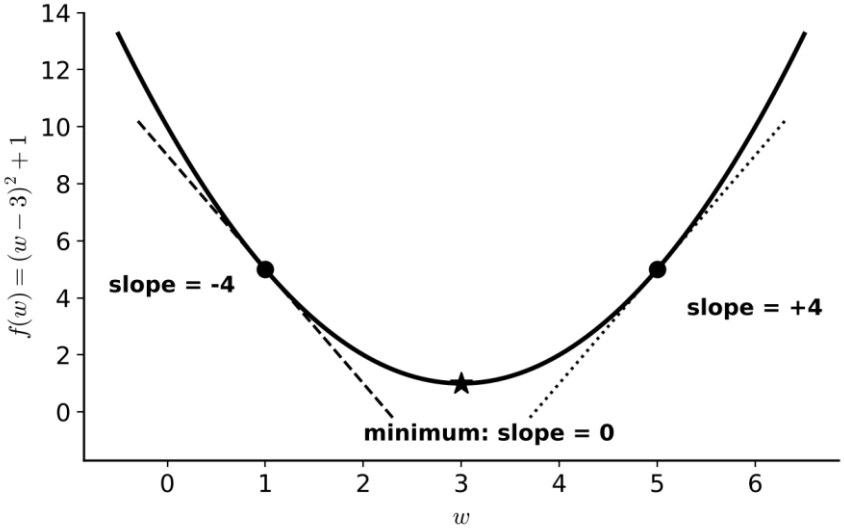


Figure: tangent slopes of f(w) = (w − 3)² + 1.

Explanation

Now calculus. The derivative of a function at a point is the slope of its tangent line there: how much the output changes for a tiny change in the input. For f(w) = (w − 3)² + 1, the derivative is 2(w − 3).

Look at the graph. At w = 1 the slope is −4: the function is decreasing, so to go down we should increase w. At w = 5 the slope is +4: the function is increasing, so to go down we should decrease w. At w = 3 the slope is zero: we are at the minimum. In both cases, moving against the sign of the slope takes us downhill.

This simple observation is the whole idea of gradient descent: compute the slope, take a step in the opposite direction, repeat. It also gives us a recipe for finding minima exactly: set the derivative to zero and solve. For this function, 2(w − 3) = 0 gives w = 3. We will use exactly this recipe to derive the best constant prediction and the least-squares solution.

Gradients and the chain rule

Slide 12

Key points

- Gradient ∇f = vector of partial derivatives
- $f = w_1^2 + 3w_2^2 \rightarrow \nabla f = (2w_1, 6w_2)$; at (1, 1): (2, 6)
- ∇f points uphill (steepest increase); −∇f points downhill
- Chain rule: $(g \circ h)' = g'(h(w)) \cdot h'(w)$
- $d/dw\ (wx - y)^2 = 2(wx - y) \cdot x$, basis of backpropagation (Week 11)

Equations

$$\nabla f(\mathbf{w}) = \left(\frac{\partial f}{\partial w_1}, \ldots, \frac{\partial f}{\partial w_d} \right)^\mathsf{T}$$

$$\frac{d}{dw}\, g(h(w)) = g'(h(w))\; h'(w)$$

Explanation

Our models have many parameters, so we need derivatives of functions of several variables. The partial derivative with respect to w_1 is the slope when we change only w_1 and hold the others fixed. Stacking all partial derivatives into a vector gives the gradient, written with the nabla symbol.

Example: for $f(w_1, w_2) = w_1^2 + 3w_2^2$, the gradient is $(2w_1, 6w_2)$. At the point (1, 1) it is (2, 6). The gradient has a beautiful property: it points in the direction of steepest increase. So minus the gradient points in the direction of steepest decrease, downhill, which is exactly where gradient descent steps.

The chain rule tells us how to differentiate compositions: the derivative of g of h of w is g-prime of h of w times h-prime of w, the outer derivative times the inner one. Example: the squared error of one prediction, $(w\,x - y)^2$, has derivative 2 (w x − y) times x. The chain rule is also the mathematical engine of backpropagation in neural networks, which we'll meet in Week 11, automatic differentiation is the chain rule applied systematically by a computer.

The matrix calculus you need today

Slide 13

Key points

- Rule 1: $\nabla(a^T w) = a$
- Rule 2: $\nabla(w^T A w) = 2Aw$ (A symmetric)
- $\|Xw - y\|^2 = w^T X^T X w - 2w^T X^T y + y^T y$
- $\Rightarrow \nabla = 2X^T Xw - 2X^T y = 2X^T(Xw - y)$
- Gradient = $X^T \times$ residuals: used for normal equations and gradient descent

Equations

$$\nabla_{\mathbf{w}} \|X\mathbf{w} - \mathbf{y}\|_2^2 = 2X^\top(X\mathbf{w} - \mathbf{y})$$

Explanation

We only need two matrix-calculus rules, and with them we can differentiate the squared error of linear regression in three lines.

Rule one: the gradient of a-transpose-w with respect to w is a. This generalises “the derivative of a·w is a”. Rule two: for a symmetric matrix A, the gradient of w-transpose-A-w is 2Aw. This generalises “the derivative of a·w² is 2a·w”.

Now expand the total squared error: the squared norm of Xw minus y equals w-transpose X-transpose X w, minus 2 w-transpose X-transpose y, plus y-transpose y. The first term is rule two with A = X-transpose-X, giving 2 X-transpose-X w. The second is rule one, giving minus 2 X-transpose y. The last term doesn't depend on w. Adding up: the gradient is 2 X-transpose times (Xw minus y).

Read this result in words: it's X-transpose times the residual vector. Each component sums, over all examples, the residual times the value of one feature. Features that correlate with the current errors get large gradient components, they need to change most. We'll use this gradient twice: set to zero for the normal equations, and step along it for gradient descent.

Convexity: when “downhill” is guaranteed to work

Slide 14

Key points

- Convex = bowl-shaped: chords lie above the graph
- Convex ⇒ every local minimum is global
- Non-convex: gradient descent can get stuck in local minima
- Linear-regression MSE, ridge and lasso objectives are convex
- Neural-network losses are non-convex (Week 11)

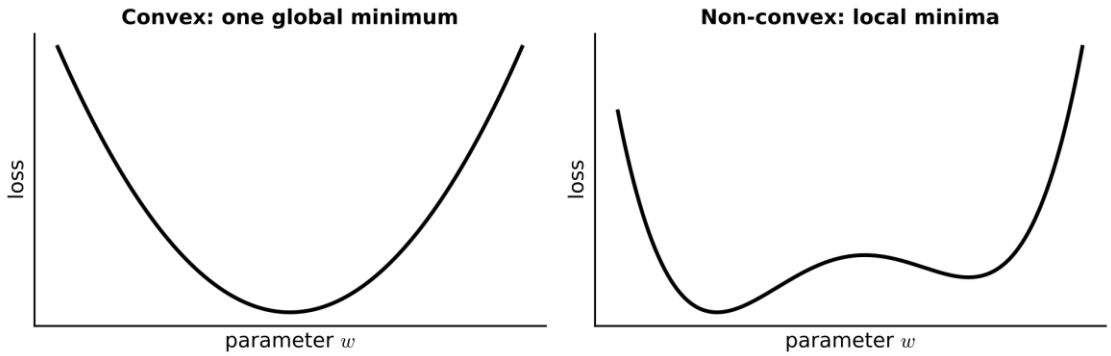


Figure: a convex and a non-convex loss.

Explanation

A function is convex if the straight line between any two points on its graph lies on or above the graph, informally, it is shaped like a bowl. Convex functions have a wonderful property: every local minimum is a global minimum. If you keep walking downhill, you cannot get stuck in a wrong valley.

The left plot is convex, the right is not: it has two valleys, and gradient descent started on the wrong side would settle in the shallower local minimum.

Good news for today: the mean squared error of linear regression is convex in the weights, it's a bowl, as we'll see in a three-dimensional plot shortly. The ridge objective is even strictly convex, so it has exactly one minimiser. The lasso objective is convex too, though not differentiable everywhere because of the absolute values. That is why linear models are so reliable: the optimisation always finds the best weights.

Neural networks, by contrast, have highly non-convex losses. That's why their training needs more care, careful initialisation, learning-rate schedules, and luck, topics for Week 11.

Probability revision: expectation, variance and noise

Slide 15

Key points

- $E[Y] = \Sigma\, y\, p(y)$ (fair die: 3.5); $Var[Y] = E[(Y - E[Y])^2]$
- Sample mean estimates $E[Y]$; more data → better estimate
- Model: $y = w^T x + \varepsilon$, noise ε with mean 0 → irreducible error
- Gaussian noise ⇒ maximum likelihood = least squares (Week 3)

Equations

$$\mathrm{E}[Y] = \sum_y y\, p(y) \qquad \mathrm{Var}[Y] = \mathrm{E}\big[(Y - \mathrm{E}[Y])^2\big]$$

$$y = \mathbf{w}^\mathsf{T}\mathbf{x} + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2) \;\Rightarrow\; \text{max likelihood} \equiv \text{min squared error}$$

Explanation

Finally, probability. The expectation of a random variable is its probability-weighted average, for a fair die, 3.5. The variance is the expected squared distance from the expectation; its square root is the standard deviation. The sample mean of observed data is our estimate of the expectation, and it gets more reliable as n grows, the law of large numbers. That principle is exactly what makes learning from data possible, as we'll see with empirical risk in a moment.

In machine learning we model the target as a deterministic part plus random noise: y equals w-transpose-x plus epsilon, where epsilon is noise with mean zero. Even the perfect model cannot predict the noise, so there's always an irreducible error.

A preview of next week: if we assume the noise is Gaussian, then choosing w to maximise the likelihood of the observed data turns out to be exactly the same as minimising the squared error. So least squares is not an arbitrary choice; it's the maximum-likelihood estimate under Gaussian noise. Next week, the same reasoning with a different distribution gives logistic regression.

Quick check: maths toolkit

Slide 16

Key points

- X (342×2) @ w (2) → (342,)
- d/dw $5w^2$ at w = 2 → 20
- (3, −4): L2 = 5, L1 = 7
- |w| convex? yes (not differentiable at 0)
- E[fair die] = 3.5
- X 442×11 → X^TX is 11×11

Explanation

Let's check the toolkit with six quick questions. Work with your neighbour for two minutes, then I'll reveal the answers one at a time.

One: X has shape 342 by 2 and w has shape 2. What is the shape of X at w? Answer: a vector of 342 predictions. Two: what is the derivative of $5w^2$ at w = 2? It's 10w, so 20. Three: for a = (3, −4), what are the L2 and L1 norms? 5 and 7. Four: is the function absolute value of w convex? Yes, it's a V shape; it's convex but not differentiable at zero, which matters for lasso. Five: the expectation of a fair six-sided die? 3.5. Six: X has shape 442 by 11, the diabetes data with an intercept column, which we'll use later. What is the shape of X-transpose-X? 11 by 11.

If you got five or six right, you're well prepared. If not, the lecture notes have a fuller revision, and the Mathematics for Machine Learning book on the reading slide is free online.

Part 2, Learning as risk minimisation

One principle behind almost every learning algorithm

Now we formalise learning itself. We describe a supervised learning problem by its ingredients, data distribution, hypothesis class, loss, define the true risk we care about and the empirical risk we can compute, and arrive at the empirical risk minimisation principle. We finish with what can go wrong, which previews the bias–variance trade-off of Week 4.

Supervised learning, formally

Slide 18

Key points

- Input space $\mathcal{X}$, output space $\mathcal{Y}$
- Unknown distribution $\mathcal{D}$ over (x, y)
- Training sample $S = \{(x_i, y_i)\}_{i=1}^{n}$ drawn i.i.d. from $\mathcal{D}$
- Hypothesis class $\boldsymbol{\mathcal{H}}$; loss $\ell(\hat{y}, y)$
- Algorithm: $S \mapsto \hat{h} \in \boldsymbol{\mathcal{H}}$
- Methods differ in $\boldsymbol{\mathcal{H}}$, ℓ and the search procedure

Explanation

Here are the ingredients of a supervised learning problem, written formally.

There is an input space, script X, for penguins, flipper lengths in millimetres, and an output space, script Y, body masses in grams. There is an unknown data distribution, D, over pairs (x, y): nature's process that generates penguins and their measurements. We never see D itself; we only see a training sample S of n pairs, assumed to be drawn independently from D, the i.i.d. assumption.

We choose a hypothesis class, script H: the set of functions our algorithm may choose from, such as all straight lines. We choose a loss function, ℓ, which says how bad a prediction is compared with the truth. And a learning algorithm takes the sample S and returns one hypothesis, h-hat, from H.

This framework covers almost everything in the course. Linear regression, logistic regression, decision trees and neural networks differ only in their hypothesis class, their loss, and how they search H. Keep this picture in mind as a checklist whenever you meet a new method.

Learning paradigms, formally

Slide 19

Key points

- Supervised: given (x, y) pairs; minimise loss(prediction, y), regression or classification
- Unsupervised: given x only; optimise a structure objective (k-means, PCA), Weeks 9–10
- Reinforcement: no dataset; maximise expected cumulative reward, Week 15

- Self-supervised: labels made from inputs (LLMs), Weeks 13–14

Explanation

In Lecture 1 we met three learning paradigms in words. Here they are in the language we just introduced, so you can see what is given and what is optimised in each.

In supervised learning we are given input–output pairs and we minimise a loss between predictions and the given outputs. Regression has a real-valued output space; classification has a finite set of classes. That's today and the next six weeks.

In unsupervised learning we are given inputs only. There is no y; instead we optimise an objective that describes structure, for example, k-means minimises the distance of points to their cluster centres, and PCA maximises the variance captured. That's Weeks 9 and 10.

In reinforcement learning, an agent is not given a dataset at all; it generates experience by acting and maximises expected cumulative reward. That's a brief overview in Week 15.

There are hybrids too, like self-supervised learning, where labels are created from the inputs themselves, the training principle behind large language models in Weeks 13 and 14. Notice that all of them are optimisation problems; what changes is what is optimised.

Problem formulation: a checklist

Slide 20

Key points

- Inputs $\mathcal{X}$ (features, units) · output $\mathcal{Y}$ (number or class)
- Hypothesis class $\boldsymbol{\mathcal{H}}$ · training loss ℓ
- Evaluation metric (may differ from ℓ) · data split
- Penguins: flipper (mm) → mass (g), lines, squared loss, RMSE/MAE on 20 % test, vs baseline
- Answer in writing before coding (lab today, proposal in Week 8)

Explanation

In Lecture 1, the Task–Experience–Performance framework was our informal tool for formulating problems. Now we can give a sharper checklist with six questions, and you should answer them in writing before writing any model code, in the lab today, and in your project proposal in Week 8.

What is an input, which features, in which units? What is the output, and is it a number or a class? What hypothesis class will we search, lines, trees, networks? What loss will training minimise? What evaluation metric matters for the application, which need not be the training loss? And how will we split the data so that our estimate of performance is honest?

The right column shows answers for today's running example: predict penguin body mass from flipper length with a straight line, trained with squared loss, evaluated with RMSE and MAE in grams on a held-out 20 percent test set, and compared with a baseline. Formulating the problem this precisely is half the work; it makes the rest of the project almost mechanical.

Loss functions: how bad is a prediction?

Slide 21

Key points

- Squared $(\hat{y} - y)^2$: smooth, standard for regression; outlier-sensitive
- Absolute $|\hat{y} - y|$: robust to outliers; kink at 0
- 0–1 loss $\mathbf{1}[\hat{y} \neq y]$: what accuracy measures; useless gradient → surrogate losses (logistic, Week 3)
- Training loss ≠ evaluation metric necessarily

Equations

$$\ell_{\text{sq}} = (\hat{y} - y)^2 \qquad \ell_{\text{abs}} = |\hat{y} - y| \qquad \ell_{0\text{-}1} = \mathbf{1}[\hat{y} \neq y]$$

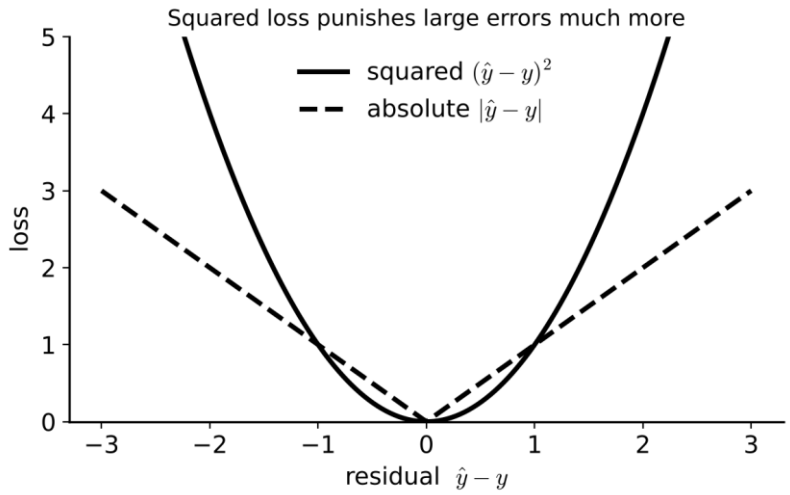


Figure: squared and absolute loss as functions of the residual.

Explanation

The loss function turns "how wrong is this prediction" into a number. Three losses matter now.

The squared loss, the prediction minus the truth, squared, is the standard for regression. It's smooth and easy to differentiate, which makes optimisation easy, and as we just said, it corresponds to Gaussian noise. But look at the plot: it grows quadratically, so one error of 3 costs as much as nine errors of 1. Squared loss is very sensitive to outliers.

The absolute loss grows only linearly, so it's more robust to outliers, but it has a kink at zero and is harder to optimise.

The zero–one loss is 1 for a wrong class and 0 for a correct one. It's what accuracy measures in classification, but it's flat almost everywhere, so its gradient is useless for optimisation. That's why classifiers are trained with smooth surrogate losses, like the logistic loss you'll meet next week.

An important distinction: the training loss and the evaluation metric need not be the same. We often train with squared loss for convenience and report MAE because it's easier to explain to non-specialists.

The true risk: what we really care about

Slide 22

Key points

- True risk = expected loss over the data distribution $\mathcal{D}$
- = generalisation error; what we really want to be small
- Cannot be computed: $\mathcal{D}$ is unknown
- Depends on the population: distribution shift changes it (Week 15)

Equations

$$R(h) = \mathrm{E}_{(\mathbf{x},\, y) \sim \mathcal{D}}[\ell(h(\mathbf{x}),\ y)]$$

Explanation

The quantity we really care about is the true risk, also called the expected loss or generalisation error. It's the average loss of a hypothesis h over the whole data distribution D, over every penguin that could ever be measured, weighted by how likely it is.

For squared loss, the true risk of a predictor is its mean squared error on the entire population. A model with low true risk will predict well on new data, which is exactly what generalisation means.

The problem: we cannot compute the true risk, because D is unknown. We only have a finite sample. Every practical method must therefore work with something we can compute and hope it approximates the true risk well.

Notice the subtle point in the definition: the expectation is over the distribution of pairs. If the deployment population differs from the one we sampled, penguins from a new island, patients from a different hospital, the true risk on the new population can differ. That's distribution shift, a topic for Week 15.

Empirical risk minimisation (ERM)

Slide 23

Key points

- Empirical risk $\hat{R}_n(h)$ = average training loss
- ERM: $\hat{h}$ = argmin over $\mathcal{H}$ of $\hat{R}_n(h)$
- Law of large numbers: $\hat{R}_n(h) \to R(h)$ as n grows (fixed h)
- Linear regression = ERM with lines + squared loss
- Training risk of the chosen $\hat{h}$ is optimistic → need a test set

Equations

$$\hat{R}_n(h) = \frac{1}{n}\sum_{i=1}^{n} \ell(h(\mathbf{x}_i),\ y_i)$$

$$\hat{h} = \arg\min_{h \in \mathcal{H}}\ \hat{R}_n(h)$$

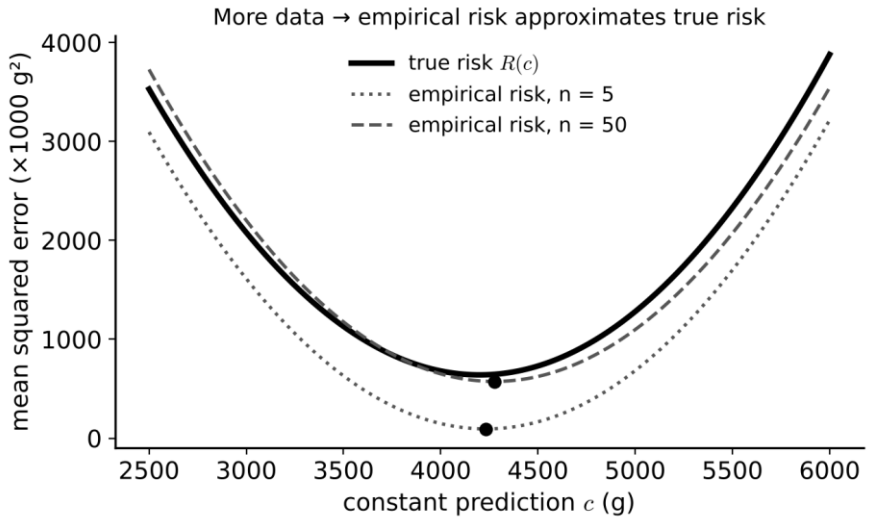


Figure: true risk and empirical risks of constant predictors for n = 5 and n = 50 (synthetic data).

Explanation

Since we can't compute the true risk, we replace the expectation by an average over the training sample. This is the empirical risk: one over n times the sum of the losses on the training examples. By the law of large numbers, for a fixed hypothesis, the empirical risk approaches the true risk as n grows.

The empirical risk minimisation principle says: choose the hypothesis in our class with the smallest empirical risk. That's it. Linear regression is ERM with lines and squared loss; logistic regression is ERM with linear classifiers and logistic loss; neural networks are ERM with networks.

The plot makes the idea concrete. The thick curve is the true risk of predicting a constant c for data with mean 4200 grams. The dotted curve is the empirical risk computed from only 5 examples; the dashed curve from 50 examples. With 50 examples, the empirical curve is close to the true one and its minimum is in nearly the right place. With 5, the curve is far off, and notice it lies below the true risk at its minimum: on the training data, the chosen model always looks better than it really is. That optimism is why we need a test set.

What can go wrong with ERM?

Slide 24

Key points

- $\mathcal{H}$ too small → approximation error → underfitting
- $\mathcal{H}$ too large (for n) → estimation error → overfitting

- Bigger $\mathcal{H}$ trades approximation for estimation error (bias–variance, Week 4)
- Remedy for estimation error: more data, smaller $\mathcal{H}$, or regularisation (Part 06)

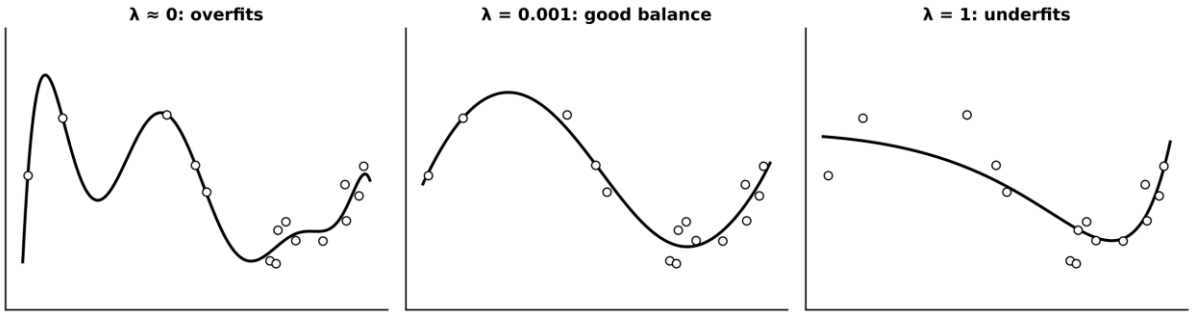


Figure: a flexible model overfits; a constrained one underfits.

Explanation

ERM is simple, but its result depends crucially on the hypothesis class, and two things can go wrong.

If H is too small, say, only constant predictions, or only straight lines for a curved relationship, then even the best hypothesis in H has high true risk. This is approximation error: the class can't express the truth. It shows up as underfitting: high error on both training and test data.

If H is too large and flexible, say, polynomials of degree 15, or a huge network with little data, ERM can find a hypothesis that fits the training noise perfectly, with near-zero empirical risk, but high true risk. This is estimation error: with finite data we can't reliably tell good hypotheses from lucky ones. It shows up as overfitting: low training error, high test error.

The total error is roughly the sum of the two, and making H bigger trades one for the other. This is the bias–variance trade-off, which gets a full lecture in Week 4. Today we'll meet one practical remedy for estimation error: regularisation, which keeps a flexible class but penalises complex hypotheses.

Part 3, Baselines

The simplest ERM, and the bar every model must clear

Before fitting any real model, we need a reference point. In this part we define baseline models, derive with ERM why the best constant prediction under squared loss is the mean, and under absolute loss the median, and compute the baseline on the penguins data. This is also the "baseline model" item of the Week 1 lab.

Why every project needs a baseline

Slide 26

Key points

- Baseline = deliberately simple model using little/no feature information
- Regression: mean or median · classification: most frequent class · time series: last value
- scikit-learn: DummyRegressor, DummyClassifier
- Can't beat the baseline → useless model or a bug
- Every lab, project and report must include a baseline

Explanation

A baseline is a deliberately simple model that uses little or no information from the features. Its purpose is to answer the question: is my model actually learning anything?

Typical baselines are: for regression, always predict the mean, or the median, of the training targets; for classification, always predict the most frequent class; for time series, predict that tomorrow equals today. Scikit-learn provides these as DummyRegressor and DummyClassifier.

A model that cannot clearly beat its baseline is useless, however sophisticated it looks. Baselines also expose bugs: if your carefully engineered model scores about the same as the mean predictor, something is probably wrong, data leakage, misaligned rows, or a forgotten intercept. And they give context to numbers: an RMSE of 380 grams sounds abstract, but "less than half the baseline's 820" is meaningful.

In this course, every lab, project and report you submit must state a baseline and compare against it. The midterm and final project rubrics reward exactly this kind of evidence-based comparison.

ERM with constants: the mean minimises squared error

Slide 27

Key points

- Hypothesis class: constants $h(x) = c$
- $d/dc\ (1/n)\Sigma(y_i - c)^2 = -(2/n)\Sigma(y_i - c) = 0 \Rightarrow c^{\times} = \bar{y}$
- Second derivative $2 > 0$ → minimum (convex)

- Absolute loss ⇒ c× = median
- Mean/median are ERM solutions, explains their outlier behaviour

Equations

$$\frac{d}{dc}\frac{1}{n}\sum_i (y_i - c)^2 = -\frac{2}{n}\sum_i (y_i - c) = 0 \;\Rightarrow\; c^* = \bar{y}$$

$$\arg\min_c \; \frac{1}{n}\sum_i |y_i - c| = \text{median}(y_1, \ldots, y_n)$$

Explanation

Let's apply ERM to the smallest interesting hypothesis class: constant predictions, h(x) = c, ignoring x entirely. Which constant c minimises the empirical squared error?

Use the recipe from the calculus revision: differentiate and set to zero. The derivative of the average of $(y_i - c)^2$ with respect to c is minus two over n times the sum of $(y_i - c)$. Setting this to zero gives the sum of y_i equals n times c, so c equals the sample mean, y-bar. The second derivative is 2, which is positive, so this is indeed a minimum, and the function is convex.

A similar argument for absolute loss shows that the best constant is the median: the derivative of the absolute value is the sign, so the optimum balances the number of points above and below c.

This is a beautiful little result. The mean and the median are not just descriptive statistics; they are the empirical risk minimisers for squared and absolute loss over constants. It also explains why the mean is pulled by outliers while the median isn't: squared loss punishes large deviations heavily, absolute loss doesn't.

The baseline on real data

Slide 28

Key points

- MSE(c) minimised at the mean 4202 g; MAE(c) at the median 4050 g
- Baseline RMSE = std of y = 801 g; baseline MAE (median) = 666 g
- Bar to beat for any model
- R^2 compares a model with this mean baseline

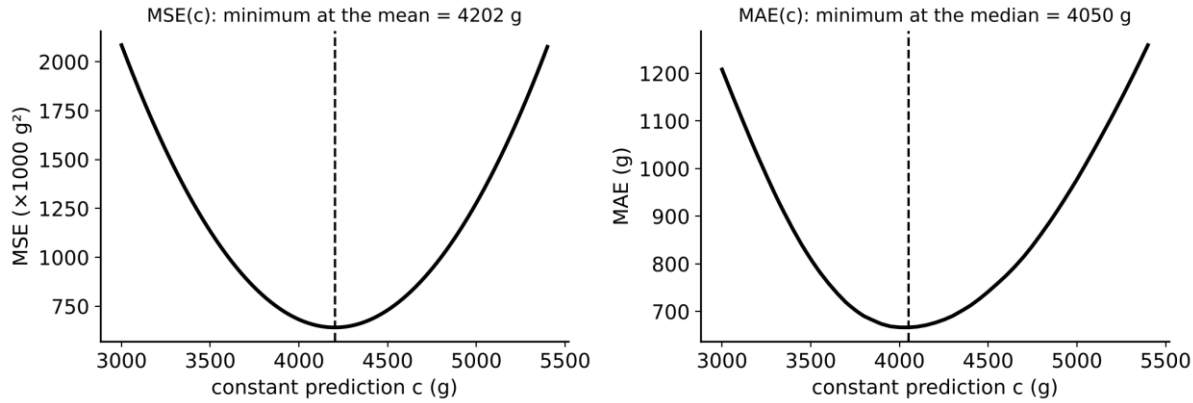


Figure: MSE(c) and MAE(c) on the penguins; minima at the mean and the median.

Explanation

Here is the derivation checked on the 342 penguins with known body mass. On the left we evaluate the mean squared error of every constant prediction between 3000 and 5400 grams. The curve is a parabola, and its minimum sits exactly at the sample mean, 4202 grams. On the right is the mean absolute error of every constant: a piecewise-linear curve whose minimum sits at the median, 4050 grams.

The values at the minima are informative too. The minimum MSE of the mean predictor is the variance of the targets, about 641 thousand grams squared, so its RMSE is 801 grams, the standard deviation of body mass. The MAE of the median predictor is about 666 grams.

These numbers are the bar to beat. Any model that uses flipper length must achieve an RMSE well below 800 grams to be worth anything. Also notice: the RMSE of the mean baseline equals the standard deviation of y. That's a handy sanity check, and it's why R-squared, which we'll define shortly, compares a model's squared error with exactly this baseline.

Baseline vs model on unseen data

Slide 29

Key points

- 80/20 split, random_state = 42; 69 test penguins
- Baseline (mean): MAE 693 g · RMSE 820 g
- Linear model: MAE 316 g · RMSE 381 g · R^2 0.78
- Same test set, same metric, otherwise no comparison
- Lab: baseline by hand in NumPy, then DummyRegressor

Code

```
from sklearn.dummy import DummyRegressor
base = DummyRegressor(strategy="mean").fit(X_train, y_train)
```

Explanation

Now the honest comparison, on unseen data. We split the penguins 80/20 into training and test sets with random_state 42. The baseline predicts the training mean for every test penguin. The linear model, which we'll derive in the next part, predicts mass from flipper length.

On the 69 test penguins, the baseline has an MAE of 693 grams and an RMSE of 820 grams. The linear model has an MAE of 316 grams and an RMSE of 381 grams, less than half the baseline's error, and an R-squared of 0.78. So one measurement, flipper length, explains most of the variation in body mass.

Two lessons. First, the baseline gives the number meaning: 381 grams alone tells you little; “53 percent less error than the mean” tells you the model has learned something real. Second, always compute both on the same test set with the same metric, or the comparison is meaningless.

The code is short: DummyRegressor and LinearRegression share the fit–predict interface from Lecture 1. In the lab, you'll compute the baseline by hand with NumPy first, then confirm it with DummyRegressor.

Part 4, Linear regression & least squares

ERM with lines and squared loss, solved exactly

Now we enlarge the hypothesis class from constants to linear functions. We write the model in vector and matrix form, define the least-squares objective, look at its convex loss surface, derive the normal equations with the matrix calculus from Part one, interpret them geometrically as a projection, compute them on real data, and discuss evaluation, the probabilistic view, and pitfalls.

The linear model

Slide 31

Key points

- $\hat{y} = wx + b$ (one feature) · $\hat{y} = w_0 + \Sigma\, w_j x_j$ (d features)
- Bias trick: add constant feature 1 → $\hat{y} = \tilde{x}^T w$
- w_j = change in $\hat{y}$ per unit of x_j, other features fixed
- Linear in the parameters: features like x^2, log x still allowed

Equations

$$\hat{y} = w\,x + b$$

$$\hat{y} = w_0 + w_1 x_1 + \cdots + w_d x_d = \tilde{\mathbf{x}}^\top \mathbf{w}$$

Explanation

Linear regression predicts the target as a weighted sum of the features plus an intercept. With one feature it's the familiar line, y-hat equals w x plus b: w is the slope and b the intercept. With d features, each feature gets its own weight.

A neat trick removes the special treatment of the intercept: add a constant feature equal to 1 to every example. Then the intercept is just the weight w_0 on that constant feature, and the whole model becomes a single dot product, x-tilde-transpose w. We'll use this “bias trick” throughout.

Interpretation is the great strength of linear models: w_j is the change in the prediction when feature j increases by one unit and all other features are held fixed. Its sign says the direction of the effect, its size the strength, in the units of the target per unit of the feature.

An important subtlety: “linear” means linear in the parameters, not necessarily in the inputs. If you add features such as x^2 or log x, the model can draw curves, yet it's still linear regression and everything today still applies. That's how the polynomial fits on the previous slides were made.

The design matrix: all examples in one equation

Slide 32

Key points

- Design matrix X: rows = examples with leading 1; columns = [1, features]
- Penguins: $X \in \mathbb{R}^{342 \times 2}$, $y \in \mathbb{R}^{342}$, $w = (w_0, w_1)$
- All predictions: $\hat{y} = Xw$
- NumPy: X = np.c_[np.ones(n), x]
- Forgetting the ones column forces the line through the origin; sklearn adds it (fit_intercept=True)

Code

```
X = np.c_[np.ones(len(x)), x]      # bias trick: (342, 2)
y_hat = X @ w
```

Explanation

With the bias trick, we stack all training examples into a design matrix X. Each row is one example with a leading 1, followed by its features. For the penguins with one feature, X has two columns: a column of ones and a column of flipper lengths. The targets go into a vector y and the parameters into a vector w with the intercept first.

Then all n predictions are a single matrix–vector product, y-hat equals X w, exactly as in the maths revision. With the first penguins: 181 millimetres, 186, 195 and so on, each row multiplied by w gives one predicted mass.

In NumPy, the column of ones is added with np.c_ or np.column_stack, one line, shown on the right. Forgetting it is the single most common mistake in from-scratch implementations: the model is then forced through the origin, which for penguins would predict zero grams for a zero-millimetre flipper and give a badly tilted line. Scikit-learn's LinearRegression adds the intercept for you through its fit_intercept parameter, which is True by default, so don't add the ones column when you use it.

The least-squares objective

Slide 33

Key points

- Linear regression = ERM with linear functions + squared loss
- $J(w) = (1/n) \| Xw - y \|^2$ = mean squared residual
- Residuals are vertical distances (errors in the target)
- Mean baseline = flat line; tilt = learning from the feature
- Least squares: Legendre & Gauss, c. 1805–1809

Equations

$$J(\mathbf{w}) = \frac{1}{n}\,\|X\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{n}\sum_{i=1}^{n}(\tilde{\mathbf{x}}_i^{\mathsf{T}}\mathbf{w} - y_i)^2$$

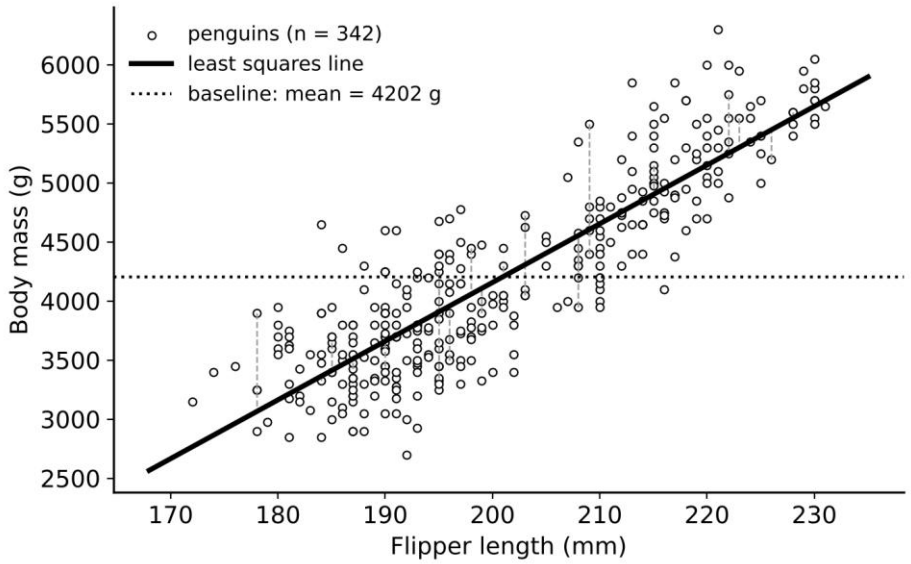


Figure: least-squares line, residuals and mean baseline on the penguins.

Explanation

Linear regression is ERM with the hypothesis class of linear functions and the squared loss. The empirical risk becomes the mean squared error, J of w: one over n times the squared norm of the residual vector Xw minus y, equivalently the average of the squared vertical distances between the points and the line.

The picture shows these residuals as dashed vertical lines for a sample of penguins. Least squares chooses the line that makes the sum of the squares of these lengths as small as possible. Note that the distances are vertical, errors in the target, not perpendicular to the line, because we only care about errors in what we predict.

The dotted horizontal line is the mean baseline from Part three. Any tilt of the line that reduces the squared residuals compared with the flat line is learning from the feature. The method goes back to Legendre and Gauss around 1805 to 1809, who used it to fit planetary and comet orbits, making least squares one of the oldest learning algorithms there is.

The loss surface is a convex bowl

Slide 34

Key points

- Each point (b, w) = one line; height = its MSE
- J is quadratic in the parameters → convex bowl, unique minimum
- Standardised feature → round bowl (circular contours)
- Raw feature → long, narrow valley (matters for gradient descent)

- Two routes to the bottom: calculus (now) or walking downhill (Part 05)

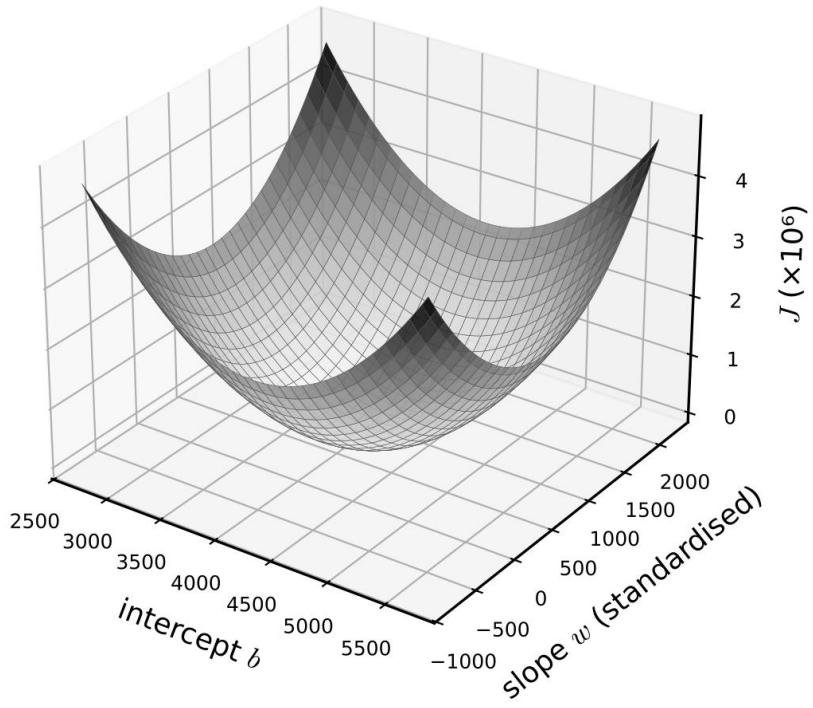


Figure: the MSE surface over (intercept, slope) with a standardised feature.

Explanation

Because the model has only two parameters with one feature, intercept and slope, we can draw J as a surface above the (b, w) plane. Each point of the plane is one candidate line; the height is its mean squared error.

The surface is a bowl: a quadratic function of the parameters, which is convex. So there is exactly one lowest point, the least-squares solution, marked by the dot, and no local minima to get stuck in. That's the convexity from Part one paying off.

For this plot we standardised the flipper length, subtracted its mean and divided by its standard deviation. With a standardised feature, the bowl is perfectly round: its contour lines are circles. With the raw feature, measured in millimetres around 200, the bowl would be extremely stretched in one direction, a long narrow valley. The shape of the bowl doesn't change where the minimum is, but it matters enormously for gradient descent, as we'll see in Part five. Keep this picture in mind: solving linear regression means finding the bottom of this bowl, either by calculus in one step, or by walking downhill.

Deriving the normal equations

Slide 35

Key points

- 1 $J(w) = (1/n)\|Xw - y\|^2$
- 2 $\nabla J(w) = (2/n)X^T(Xw - y)$

- 3 Set ∇J = 0 ⇒ $X^TXw = X^Ty$ (normal equations)
- 4 If X^TX invertible: $\hat{w} = (X^TX)^{-1}X^Ty$
- One feature: slope = cov(x, y)/var(x); line passes through $(\bar{x}, \bar{y})$

Equations

$$J(\mathbf{w}) = \frac{1}{n}\,\|X\mathbf{w} - \mathbf{y}\|_2^2 = \frac{1}{n}\sum_{i=1}^{n}(\tilde{\mathbf{x}}_i^{\top}\mathbf{w} - y_i)^2$$

$$\nabla J(\mathbf{w}) = \frac{2}{n}\,X^{\top}(X\mathbf{w} - \mathbf{y})$$

$$X^{\top}X\,\hat{\mathbf{w}} = X^{\top}\mathbf{y} \quad \Rightarrow \quad \hat{\mathbf{w}} = (X^{\top}X)^{-1}X^{\top}\mathbf{y}$$

$$\hat{w} = \frac{\sum_i (x_i - \bar{x})(y_i - \bar{y})}{\sum_i (x_i - \bar{x})^2}, \qquad \hat{b} = \bar{y} - \hat{w}\,\bar{x}$$

Explanation

Now we find the bottom of the bowl exactly. Step one: write the objective, J of w equals one over n times the squared norm of Xw minus y. Step two: take the gradient, using the matrix calculus from Part one: two over n times X-transpose times (Xw minus y). Step three: at the minimum of a convex differentiable function the gradient is zero. The two over n disappears, and we get X-transpose-X w equals X-transpose y. These are the normal equations: a system of d linear equations in d unknowns.

Step four: if X-transpose-X is invertible, full rank, as discussed, the unique solution is w-hat equals the inverse of X-transpose-X times X-transpose y.

That's the complete derivation, in four lines. For one feature it reduces to the textbook formulas: the slope is the covariance of x and y divided by the variance of x, and the intercept makes the line pass through the point of means, x-bar, y-bar.

Remember the computational advice: in code we solve the system with np.linalg.solve rather than forming the inverse.

Least squares on the penguins

Slide 36

Key points

- Solution: intercept −5781 g, slope 49.7 g/mm
- +1 mm flipper ↔ ≈ +50 g mass
- Predictions: 200 mm → 4156 g · 220 mm → 5150 g
- Intercept is meaningless (0 mm flipper): don't extrapolate

- Association, not causation

Equations

$$\widehat{\text{mass}} \approx -5781 + 49.7 \times \text{flipper}$$

Explanation

Let's compute the solution on all 342 penguins. Solving the normal equations gives an intercept of −5781 grams and a slope of 49.7 grams per millimetre.

Interpretation: each extra millimetre of flipper length is associated with about 50 grams more body mass. A penguin with a 200-millimetre flipper is predicted to weigh about 4156 grams; one with 220 millimetres about 5150 grams.

What about the intercept, −5781 grams? Literally, it's the predicted mass of a penguin with a zero-length flipper, physically meaningless. That's fine: the intercept simply positions the line where we have data, between 172 and 231 millimetres. But it's a warning about extrapolation: a linear model is only trustworthy within the range of the training data. Predicting for a 100-millimetre flipper would give a negative mass.

Also remember the word “associated”. The slope describes how mass and flipper length vary together in this population; it doesn't say that growing a longer flipper causes a penguin to gain weight. Causal claims need more than a regression, a recurring theme for responsible ML in Week 15.

Geometry: least squares is a projection

Slide 37

Key points

- $y \in \mathbb{R}^n$; all predictions Xw lie in the column space of X (a plane)
- Least squares = closest point = orthogonal projection $\hat{y}$
- Normal equations $\Leftrightarrow X^T(y - \hat{y}) = 0$: residual $\perp$ every column
- Ones column $\Rightarrow$ residuals sum to zero

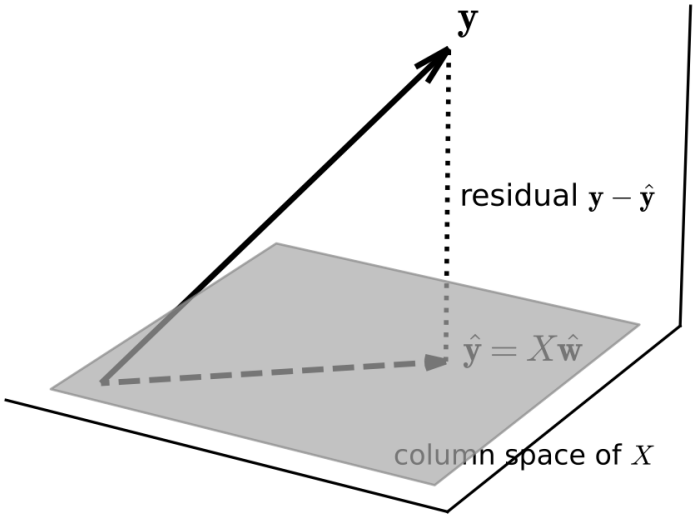


Figure: ŷ is the orthogonal projection of y onto the column space of X.

Explanation

There's a beautiful geometric picture behind the normal equations. Think of the target vector y as a point in n-dimensional space, one coordinate per training example. Every possible prediction vector Xw is a combination of the columns of X, so all of them lie in a flat subspace: the column space of X, the grey plane in the picture.

Least squares finds the point in that plane closest to y. The closest point is the orthogonal projection: drop a perpendicular from y onto the plane. The foot of the perpendicular is y-hat, our vector of fitted values, and the perpendicular itself is the residual vector, y minus y-hat.

Now read the normal equations again: X-transpose times (y minus X w-hat) equals zero. That says the residual is orthogonal to every column of X. So the normal equations are literally the statement "the residual is perpendicular to the plane". One consequence: because the column of ones is in X, the residuals are orthogonal to it, they sum to zero. The fitted line never systematically over- or under-predicts on the training data.

You don't need to visualise 342 dimensions; the three-dimensional picture captures everything.

Computing the solution: three ways, one answer

Slide 38

Key points

- np.linalg.solve(X.T@X, X.T@y), direct normal equations
- np.linalg.lstsq(X, y), stable; handles singular X^TX

- LinearRegression(), intercept handled; library integration
- All give −5780.83 and 49.69 on penguins: test from-scratch code against a library
- Cost ≈ O(nd² + d³): fine for small d; impossible for huge d; no closed form for lasso/logistic

Code

```
w1 = np.linalg.solve(X.T @ X, X.T @ y)
w2, *_ = np.linalg.lstsq(X, y, rcond=None)
lr = LinearRegression().fit(x.reshape(-1, 1), y)   # all: -5780.83, 49.69
```

Explanation

In practice there are three ways to compute the least-squares weights, and they must agree.

First, by solving the normal equations with np.linalg.solve, the direct translation of our derivation. Second, with np.linalg.lstsq, which solves the least-squares problem directly with a numerically stable factorisation; it also copes when X-transpose-X is singular. Third, with scikit-learn's LinearRegression, which handles the intercept for you and integrates with the rest of the library.

On the penguins all three give an intercept of −5780.83 and a slope of 49.69. Checking that your from-scratch result matches a trusted library to several decimal places is the standard way to test an implementation, you'll do exactly that in the lab.

What does it cost? Forming X-transpose-X takes on the order of n times d squared operations, and solving the d-by-d system on the order of d cubed. For thousands of examples and tens of features that's instant. But with a million features, d cubed becomes impossible, and some models, like lasso and logistic regression, have no closed-form solution at all. That's the motivation for gradient descent.

Evaluating a regression model: R², MAE and RMSE

Slide 39

Key points

- $R^2 = 1 - SS_{res} / SS_{tot}$; SS_{tot} = squared error of the mean baseline
- $R^2 = 1$ perfect · 0 = baseline · < 0 worse than baseline
- Penguins: train R^2 0.759 (RMSE 393 g, MAE 313 g) · test R^2 0.782
- Report an error in target units + R^2 + the baseline

Equations

$$R^2 = 1 - \frac{\sum_i (y_i - \hat{y}_i)^2}{\sum_i (y_i - \bar{y})^2}$$

Explanation

How good is the fit? We already used MAE and RMSE, which are in the units of the target. A third, unit-free metric is the coefficient of determination, R-squared.

R-squared is one minus the ratio of the model's sum of squared residuals to the sum of squared deviations from the mean. The denominator is exactly the squared error of the mean baseline from Part three. So R-squared measures the fraction of the baseline's squared error that the model removes. R-squared of 1 is a perfect fit; 0 means no better than the mean; negative means worse than the mean, which can happen on test data for a bad model.

On the penguins training data, R-squared is 0.759: flipper length explains about 76 percent of the variance in body mass. On the test set it's 0.782. The training RMSE is 393 grams and MAE 313 grams.

Which metric should you report? RMSE when large errors matter more; MAE when you want a robust, easy-to-explain average error; R-squared for a unit-free summary. In this course, report at least one error metric in the target's units alongside R-squared, and always the baseline.

Assumptions and pitfalls of linear regression

Slide 40

Key points

- Non-linearity → check residual plots; add features
- Outliers → squared loss is sensitive
- Extrapolation → only trust the training range
- Collinearity → unstable coefficients (ridge helps)
- Compare coefficients only on standardised features
- Correlation ≠ causation

Explanation

Linear regression is simple and interpretable, but it has blind spots you must check for.

Non-linearity: if the true relationship curves, a straight line underfits. Plot the residuals against the predictions; a visible pattern means the model is missing structure, add features or use a more flexible model. Outliers: squared loss makes the fit very sensitive to a few extreme points, a single outlier can tilt the whole line. Extrapolation: predictions outside the training range are unreliable. Collinearity: when features are strongly correlated, X-transpose-X is nearly singular and the individual coefficients become unstable and hard to interpret, even though predictions may be fine;

ridge regression fixes the instability. Different scales: coefficients can only be compared across features if the features are standardised. And finally, correlation is not causation.

None of these mean you shouldn't use linear regression. It's the first model to try for any regression problem, and an excellent baseline for more complex ones. They mean you should look at residual plots and think about the data before trusting the coefficients.

Part 5, Gradient descent

Walking downhill when there is no formula, or the formula is too expensive

The normal equations solved linear regression in one step. But most models in this course have no closed-form solution, and even linear regression becomes expensive with many features. Gradient descent is the general-purpose alternative: start somewhere, compute the gradient, take a small step downhill, repeat. In this part we see the algorithm, two animations, the role of the learning rate and of feature scaling, stochastic and mini-batch variants, and a from-scratch implementation.

Why iterative optimisation?

Slide 42

Key points

- Closed form ≈ $O(nd^2 + d^3)$; one GD step ≈ $O(nd)$
- No closed form: lasso, logistic regression (Week 3), neural networks (Week 11)
- Huge or streaming data → stochastic/mini-batch updates
- GD variants (momentum, Adam) train all modern neural networks
- Linear regression: we know the answer, so we can check GD

Explanation

Why not always use the closed form? Three reasons.

First, cost. Solving the normal equations costs on the order of n d^2 plus d^3 operations. With d in the millions, common in text and image problems, d cubed is out of reach. One gradient-descent step costs only on the order of n times d.

Second, many models have no closed form at all. Lasso, because of the absolute values; logistic regression next week, because the sigmoid makes the equations non-linear; and neural networks. For all of them, the only general way to minimise the empirical risk is iterative.

Third, data that doesn't fit in memory, or that arrives as a stream. Stochastic gradient descent can learn from one example, or a small batch, at a time.

That's why gradient descent and its descendants, momentum, Adam and others, are the workhorses of modern machine learning. Every large neural network, including the language models of Week 14, is trained by a variant of the algorithm on the next slide. Linear regression is the perfect place to learn it, because we know the exact answer and can check that gradient descent finds it.

The gradient descent algorithm

Slide 43

Key points

- Initialise w (zeros or small random)

- Repeat: $w \leftarrow w - \eta \nabla J(w)$, with $\nabla J(w) = (2/n)X^T(Xw - y)$
- η = learning rate (step size)
- Small enough η + convex J ⇒ converges to the least-squares solution
- Choices: learning rate and stopping rule

Equations

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$$

$$\nabla J(\mathbf{w}) = \frac{2}{n} X^\top (X\mathbf{w} - \mathbf{y})$$

Explanation

Here is the algorithm. Start from some initial parameters, zeros or small random numbers. Then repeat: compute the gradient of the objective at the current parameters, and move the parameters a small step in the opposite direction. The step size is controlled by the learning rate, η, eta.

For linear regression the gradient is the formula we derived: two over n times X-transpose times the residuals. So each step costs one prediction, X w, one residual vector, and one product with X-transpose, all vectorised in NumPy.

Why does it work? The negative gradient is the direction of steepest local decrease. For a small enough step, J decreases at every iteration; and because J is convex, there are no bad local minima to stop at, so we converge to the least-squares solution.

The pseudocode on the right is essentially the real code, about six lines of NumPy, which you'll write in the lab. Notice the two choices you must make: the learning rate and when to stop. We'll discuss both in a moment, after watching the algorithm in action.

Video: gradient descent fitting the penguins

Slide 44

Key points

- Left: data and current line · right: path on the loss contours
- Start far away (MSE in the millions)
- Steps shrink automatically as the gradient shrinks
- After 70 steps: MSE 154,546 = closed-form solution
- Straight path because the feature is standardised (round bowl)

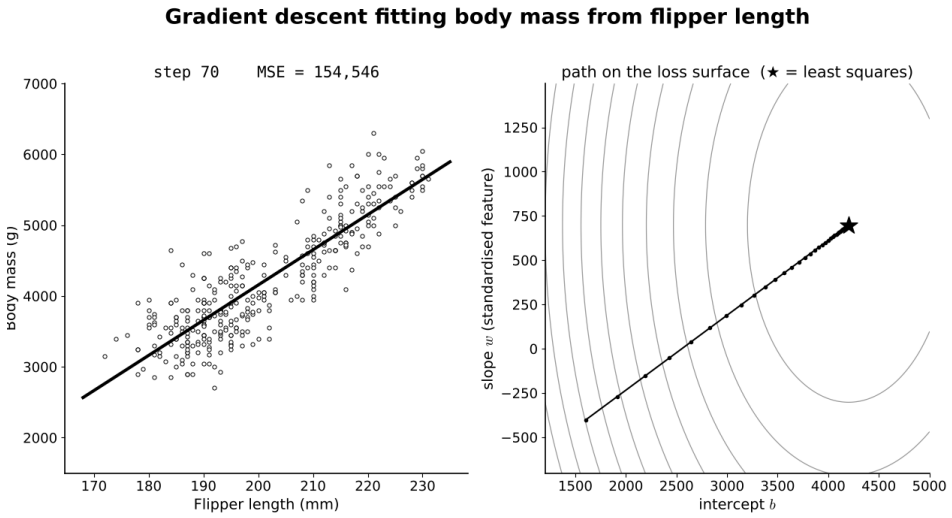


Figure: final frame of the gradient-descent animation.

Explanation

Click to play. On the left you see the penguins and the current line; on the right, the parameters on the loss surface, drawn as contour lines, with the least-squares solution marked by a star.

We start deliberately far away, a line that is too low and too flat, with a mean squared error in the millions. Each step moves the parameters along the negative gradient, and you can watch the line rise and rotate towards the data. The error printed above the left panel falls quickly at first, then more and more slowly, because the gradient gets smaller as we approach the bottom of the bowl. Steps are big when the slope is steep and small when it's gentle, automatically.

After 70 steps the MSE is 154,546, matching the closed-form solution, and the line is exactly the least-squares line we computed with the normal equations.

Notice the path on the right is a straight line towards the star. That's because we standardised flipper length, so the bowl is round and the gradient points straight at the minimum. Keep that in mind for the feature-scaling slide.

The learning rate: the most important hyperparameter

Slide 45

Key points

- Too small (0.005): very slow
- Good (0.05, 0.4): fast, smooth decrease
- Too large (1.02): overshoots, diverges → nan
- Safe η depends on the curvature (here $\eta < 1$)
- Recipe: try a log grid, plot loss vs iteration, take the largest smooth one

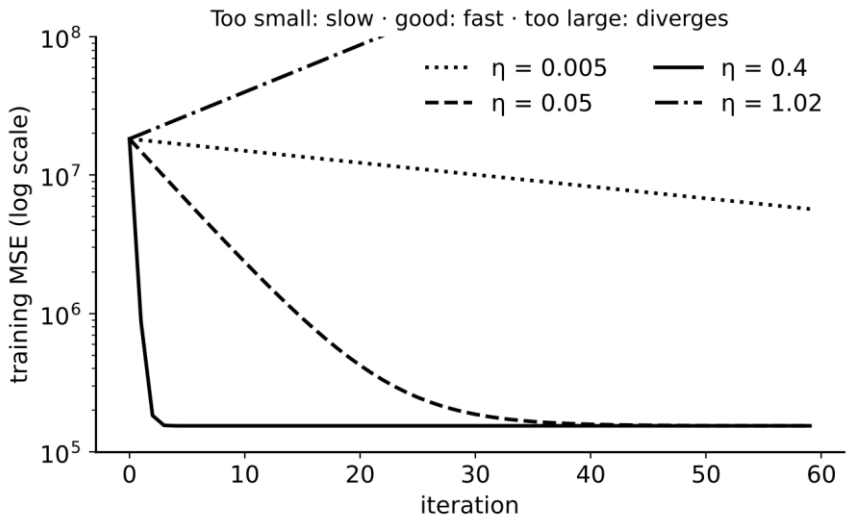


Figure: training loss for four learning rates (log scale).

Explanation

The learning rate η sets how far each step goes, and it can make or break gradient descent.

Too small, and progress is painfully slow: in the plot, with η = 0.005, the training error has barely moved after 60 iterations. Well chosen, η = 0.05 or 0.4 here, and the error drops to the minimum within a few dozen steps. Too large, η = 1.02, and each step overshoots the bottom of the bowl by more than it gains; the error grows with every step, the algorithm diverges. In practice you'd see the loss explode and then become “nan” in NumPy, not a number.

For linear regression on a standardised feature there's a precise threshold: steps above η = 1 diverge here, because the curvature of this bowl is 2. In general, the safe learning rate depends on the curvature of the loss, which depends on the scaling of the features, another reason to standardise.

The practical recipe: try values on a logarithmic grid, 0.001, 0.01, 0.1, 1, plot the training loss against the iteration for each, and pick the largest value that decreases smoothly. Always plot the loss curve; it's the single most useful diagnostic of any training run.

Video: three learning rates on one bowl

Slide 46

Key points

- Same bowl $f(w) = w^2$, same start $w = -2.4$
- $\eta = 0.05$: slow ($w = -0.55$ after 14 steps)
- $\eta = 0.35$: fast convergence
- $\eta = 1.08$: bounces outward, diverges ($w \approx -19$)
- Update $w \leftarrow (1 - 2\eta)w$: converges iff $|1 - 2\eta| < 1$

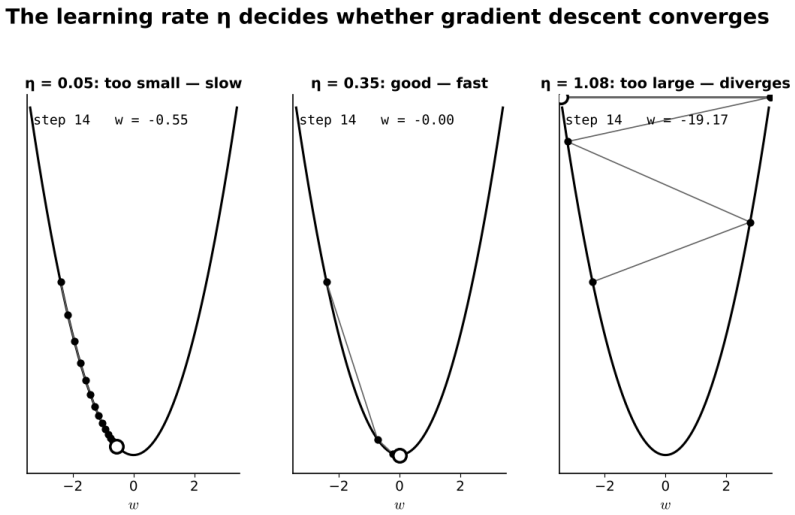


Figure: final frame of the learning-rate animation.

Explanation

Click to play. All three panels minimise the same simple bowl, $f(w) = w^2$, starting from the same point, $w = -2.4$. Only the learning rate differs.

On the left, $\eta = 0.05$: each step moves only 10 percent of the remaining distance, so after 14 steps we're still at −0.55, slow but steady. In the middle, $\eta = 0.35$: each step cuts the distance by more than half, and we reach the minimum almost immediately. On the right, $\eta = 1.08$: each step jumps over the minimum to the other side and lands higher than before. The points bounce from side to side, further out each time, and after 14 steps w is about −19. That's divergence.

Here's the arithmetic behind it. For $f(w) = w^2$, the update is $w \leftarrow w - \eta \cdot 2w = (1 - 2\eta)w$. If the factor $1 - 2\eta$ has absolute value less than 1, w shrinks to zero; if it's greater than 1, w grows without limit. So η must be between 0 and 1 for this bowl. Steeper bowls need smaller learning rates. That simple multiplier is the whole story of learning-rate stability for quadratic losses.

Feature scaling: round bowls are easy

Slide 47

Key points

- Different scales → narrow valley → zig-zag, small η, slow
- Standardise: $x' = (x - \mu)/\sigma$ → round bowl, direct path
- Fit μ, σ on the training set only (leakage, Week 5)
- Convert coefficients back for interpretation
- Needed for gradient descent and for regularisation

Equations

$$x_j' = \frac{x_j - \mu_j}{\sigma_j}$$

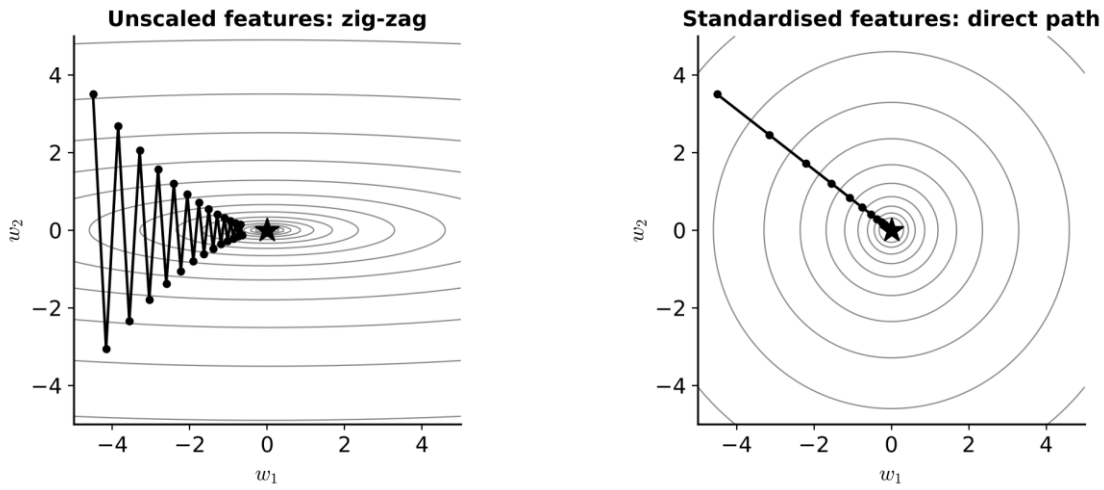


Figure: gradient descent on unscaled (left) and standardised (right) features.

Explanation

When features have very different scales, say, flipper length in millimetres around 200 and bill depth around 17, or income in dollars next to age in years, the loss surface becomes a long, narrow valley: very steep across, very flat along.

The left plot shows what gradient descent does in such a valley. The gradient points mostly across the valley, not along it, so the parameters zig-zag from wall to wall and crawl towards the minimum. And the steep direction forces a small learning rate, which makes progress along the flat direction even slower.

The fix is standardisation: subtract each feature's mean and divide by its standard deviation, so every feature has mean 0 and standard deviation 1. The bowl becomes round, as on the right, and gradient descent goes almost straight to the minimum with a larger learning rate.

Two practical rules. First, compute the mean and standard deviation on the training set only, and apply the same numbers to the test set, computing them on all the data leaks test information into training, a form of data leakage you'll study in Week 5. Second, remember to transform predictions back, or convert the coefficients, if you need them in original units. Closed-form least squares doesn't need scaling; gradient descent and regularisation do.

Batch, mini-batch and stochastic gradient descent

Slide 48

Key points

- Batch: gradient over all n, exact, expensive
- SGD: one random example, cheap, noisy, jitters near the minimum
- Mini-batch (16–512): the practical compromise

- Decreasing η (schedule) lets SGD settle
- Epoch = one pass through the training set

Equations

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla J(\mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \frac{1}{|B|} \sum_{i \in B} \nabla \ell_i(\mathbf{w})$$

$$\mathbf{w} \leftarrow \mathbf{w} - \eta \nabla \ell_i(\mathbf{w}) \quad \text{(one random example } i\text{)}$$

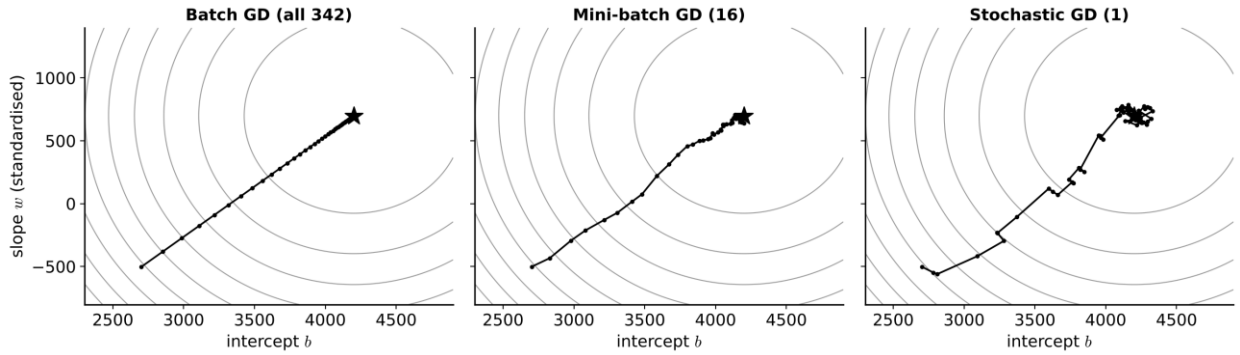


Figure: 60 steps of batch, mini-batch and stochastic gradient descent on the penguins.

Explanation

So far each step used the gradient over all n training examples. That's batch gradient descent: accurate steps, but each costs a full pass through the data.

Stochastic gradient descent, SGD, estimates the gradient from a single random example per step. Each step is n times cheaper but noisy: the path on the right wanders and keeps jittering around the minimum, because single examples disagree. Mini-batch gradient descent is the compromise used in practice: average the gradient over a small random batch, typically 16 to 512 examples. The middle path is much smoother than SGD and each step is still cheap.

The noise isn't only a cost. For large datasets, many cheap noisy steps beat a few expensive exact ones, and in neural networks the noise even helps escape poor regions of non-convex losses. To make SGD settle exactly at the minimum, the learning rate is usually decreased over time, a learning-rate schedule.

A pass through the whole training set is called an epoch. All three panels show 60 steps with the same learning rate on the penguins data, but batch GD used 342 examples per step, mini-batch 16 and SGD just one.

When to stop, and gradient descent vs the normal equations

Slide 49

Key points

- Stop: max iterations · gradient norm < tol · loss change < tol · early stopping on validation

- Early stopping = regularisation (Week 4)
- Normal equations: exact, no hyperparameters, $O(d^3)$, linear regression only
- GD: approximate, needs η, scaling, stopping; cheap steps; general

Explanation

When should gradient descent stop? Common stopping rules: a fixed maximum number of iterations or epochs; the gradient's norm falling below a small tolerance, meaning we're at a flat point, for a convex loss, the minimum; the training loss changing by less than a tolerance between iterations; or, most importantly in practice, early stopping, where we monitor the error on a separate validation set and stop when it starts to rise. Early stopping is itself a form of regularisation, which we'll analyse in Week 4.

The table compares the two routes to the least-squares solution. The normal equations give the exact answer in one shot, with no hyperparameters and no need for scaling, but cost grows with d cubed and they exist only for this special problem. Gradient descent needs a learning rate, a stopping rule and scaled features, and gives an approximate answer, but each step is cheap, it works for huge d and for streaming data, and it applies to almost any differentiable loss.

For small linear-regression problems, use the closed form or scikit-learn. For everything from next week onwards, you'll need gradient descent.

Gradient descent from scratch in NumPy

Slide 50

Key points

- Standardise → design matrix → w = 0 → 200 steps, $\eta = 0.1$
- Result (standardised): intercept 4201.75 (= mean mass), slope 697.64
- Back to mm: slope 697.64 / 14.04 = 49.69 g/mm; intercept −5780.8
- Matches the normal equations exactly
- Validate: known answer, library, loss curve

Code

```
x_s = (x - x.mean()) / x.std()
X = np.c_[np.ones(n), x_s]
w = np.zeros(2); eta = 0.1
for step in range(200):
    grad = 2 / n * X.T @ (X @ w - y)
    w = w - eta * grad
slope = w[1] / x.std()                  # 49.69 g/mm
intercept = w[0] - slope * x.mean()     # -5780.8
```

Explanation

Here is a complete from-scratch implementation for the penguins, the core of today's lab.

We standardise flipper length, build the design matrix with a column of ones, start from zero weights, and run 200 steps with learning rate 0.1, each step computing the gradient and moving against it. The result, in standardised units, is an intercept of 4201.75, which is exactly the mean body mass, since the standardised feature has mean zero, and a slope of 697.64 grams per standard deviation of flipper length.

To compare with the closed-form solution in original units, convert back. The slope per millimetre is 697.64 divided by the standard deviation of flipper length, 14.04 millimetres, giving 49.69 grams per millimetre. The intercept is 4201.75 minus that slope times the mean flipper length, 200.9 millimetres, giving −5780.8. Both match the normal equations.

Always validate from-scratch code this way: against a known answer, against a library, and by plotting the loss curve. In the lab you'll also record the loss at every step and experiment with different learning rates and with unscaled features.

Part 6, Regularisation: ridge & lasso

Trading a little training error for much better generalisation

ERM can overfit when the hypothesis class is flexible relative to the amount of data. Regularisation adds a penalty on the size of the weights to the empirical risk. In this part we meet ridge regression with the L2 penalty and its closed form, see it as weight decay in gradient descent, meet lasso with the L1 penalty and its sparsity, understand the difference geometrically with an animation, and compare them on a real medical dataset.

Why regularise?

Slide 52

Key points

- Flexible model + little data → overfitting with huge, cancelling weights
- Regularised objective = training error + λ × penalty(w)
- $\lambda = 0$: plain ERM (overfits) · large λ: underfits · well-chosen λ: generalises
- Also fixes collinearity (better-conditioned matrix)
- Same idea as weight decay in neural networks (Week 11)

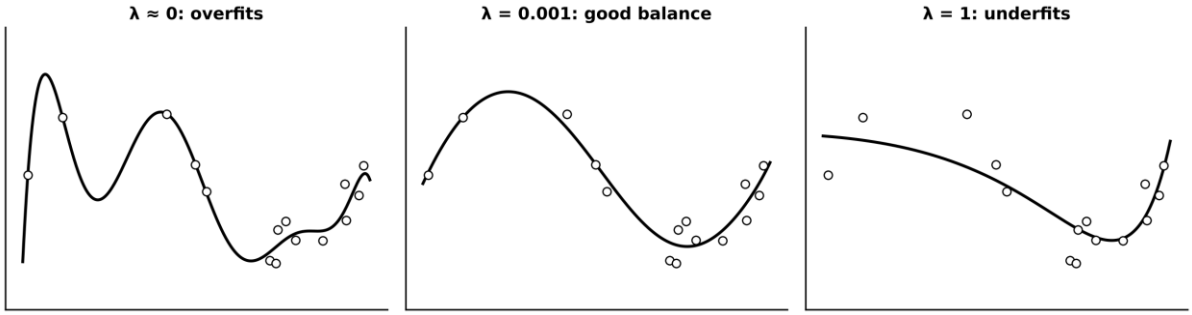


Figure: degree-12 polynomial with ridge penalties $\lambda \approx 0$, 0.001 and 1.

Explanation

Here's the problem regularisation solves. The three plots fit the same 15 noisy points from a sine curve with a degree-12 polynomial, a very flexible linear model, with 12 polynomial features. With no penalty, on the left, ERM finds wild curves that pass close to every point: overfitting. Such fits always have very large coefficients that nearly cancel each other out.

Regularisation changes the objective: training error plus λ times a penalty on the size of the weights. The penalty says, “all else equal, prefer small weights”, which means smoother, simpler functions. With a well-chosen λ, in the middle, the curve follows the real pattern. With too large a λ, on the right, the weights are forced so small that the model underfits.

So λ is a dial between the two failure modes from Part two: $\lambda = 0$ is plain ERM with its estimation error; large λ shrinks the effective hypothesis class towards a constant, adding approximation error. Regularisation also fixes the collinearity problem: with a penalty, the matrix to invert becomes well

conditioned. And it's not just for linear models: weight decay in neural networks, which we'll meet in Week 11, is the same idea.

Ridge regression (L2 penalty)

Slide 53

Key points

- J_ridge = ‖Xw − y‖² + λ‖w‖²: shrinks weights, rarely to exactly 0
- Closed form ŵ = (XᵀX + λI)⁻¹Xᵀy, invertible for any λ > 0
- Don't penalise the intercept
- Standardise features first
- Choose λ on validation data; sklearn: Ridge(alpha=λ)

Equations

$$J_{\text{ridge}}(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|_2^2 + \lambda \|\mathbf{w}\|_2^2$$

$$\hat{\mathbf{w}}_{\text{ridge}} = (X^\top X + \lambda I)^{-1} X^\top \mathbf{y}$$

Code

```
A = Xs.T @ Xs + lam * np.eye(d)          # Xs standardised, yc centred
w = np.linalg.solve(A, Xs.T @ yc)
Ridge(alpha=lam).fit(X_train_s, y_train)  # same coefficients
```

Explanation

Ridge regression adds the squared L2 norm of the weights to the sum of squared errors, multiplied by λ. Large weights are penalised quadratically, so all weights are pulled towards zero, shrunk, but they almost never become exactly zero.

Ridge keeps the closed-form advantage. Repeating our derivation with the extra term, the gradient gets an additional 2λw, and setting it to zero gives X-transpose-X plus λ times the identity, times w, equals X-transpose y. Adding λ to the diagonal makes the matrix invertible for any λ greater than zero, even with collinear features or more features than examples, which solves the instability problem from Part four.

Three practical rules. First, do not penalise the intercept: we want to shrink the effects of features, not the average level of the predictions. Scikit-learn's Ridge handles this for you. Second, standardise the features first: the penalty treats all weights equally, so their scales must be comparable, otherwise a feature measured in millimetres is penalised differently from the same feature in metres. Third, choose λ on validation data, never on the test set. In scikit-learn the parameter is called alpha, and its objective matches the formula on this slide exactly.

Ridge in gradient descent: weight decay

Slide 54

Key points

- $\nabla J_{ridge} = \nabla J + 2\lambda w$
- Update: $w \leftarrow (1 - 2\eta\lambda)w - 2\eta X^T(Xw - y)$
- Each step shrinks (decays) every weight, then data pushes back
- Called weight decay in neural networks (Week 11)
- Early stopping from $w = 0$ has a similar shrinking effect

Equations

$$\mathbf{w} \leftarrow (1 - 2\eta\lambda)\,\mathbf{w} - 2\eta\, X^{\top}(X\mathbf{w} - \mathbf{y})$$

Explanation

What does the ridge penalty do inside gradient descent? Differentiate the ridge objective: the gradient is the usual least-squares gradient plus $2\lambda w$. Substituting into the update rule and collecting the terms in w gives the expression on the slide: w becomes $(1 - 2\eta\lambda)$ times w, minus the usual data-driven step.

Read it in two parts. First, before each step, every weight is multiplied by a number slightly less than one, it decays a little towards zero. Second, the data pushes the weights in the direction that reduces training error. The final weights balance the two: weights survive only if the data keeps pushing them up. That's why ridge is called weight decay in the neural-network literature, where it's one of the most common regularisers, you'll see it again in Week 11.

This view also gives intuition for early stopping, which we mentioned earlier: starting from zero and stopping gradient descent early keeps weights small, much as weight decay does. Two different mechanisms, similar effect, and a hint that regularisation is a broader idea than adding a penalty term.

Lasso regression (L1 penalty)

Slide 55

Key points

- $J_{lasso} = (1/2n)\|Xw - y\|^2 + \alpha\|w\|_1$ (scikit-learn's exact objective)
- Sets weights exactly to 0 → sparse model, automatic feature selection
- Useful with many candidate features and few relevant ones
- Convex but not differentiable at 0 → no closed form; coordinate descent
- Standardise; don't penalise intercept; choose α on validation data
- Tibshirani (1996)

Equations

$$J_{\text{lasso}}(\mathbf{w}) = \frac{1}{2n}\|X\mathbf{w} - \mathbf{y}\|_2^2 + \alpha \|\mathbf{w}\|_1$$

Code

```
lasso = Lasso(alpha=0.1).fit(X_train, y_train)
(lasso.coef_ != 0).sum()              # number of features kept
```

Explanation

The lasso, least absolute shrinkage and selection operator, introduced by Robert Tibshirani in 1996, replaces the squared L2 penalty by the L1 norm, the sum of absolute values of the weights. The formula on the slide is the exact objective used by scikit-learn's Lasso, with the squared error scaled by one over 2n and the penalty strength called alpha.

The change looks small, but the effect is qualitatively different. Lasso doesn't just shrink weights; as alpha grows, it sets more and more of them exactly to zero. The fitted model uses only a subset of the features, lasso performs automatic feature selection, producing sparse, more interpretable models. That's valuable when you have many candidate features and expect only a few to matter, as in genetics, text, or medical data.

The price: the absolute value has a kink at zero, so the objective is convex but not differentiable there, and there's no closed-form solution. Lasso is solved by specialised algorithms, typically coordinate descent, which optimises one weight at a time and can set it exactly to zero. As with ridge: standardise features first, don't penalise the intercept, and choose alpha on validation data.

Why lasso gives zeros: the geometry

Slide 56

Key points

- Equivalent constrained form: min error s.t. $\|w\| \le t$
- Ridge region = disc (L2 ball); lasso region = diamond (L1 ball) with corners on the axes
- Solution = where the smallest error contour first touches the region
- Disc: smooth contact → both weights ≠ 0 (≈ 0.84, 0.54)
- Diamond: contact at a corner → $w_2 = 0$ exactly (1.2, 0)

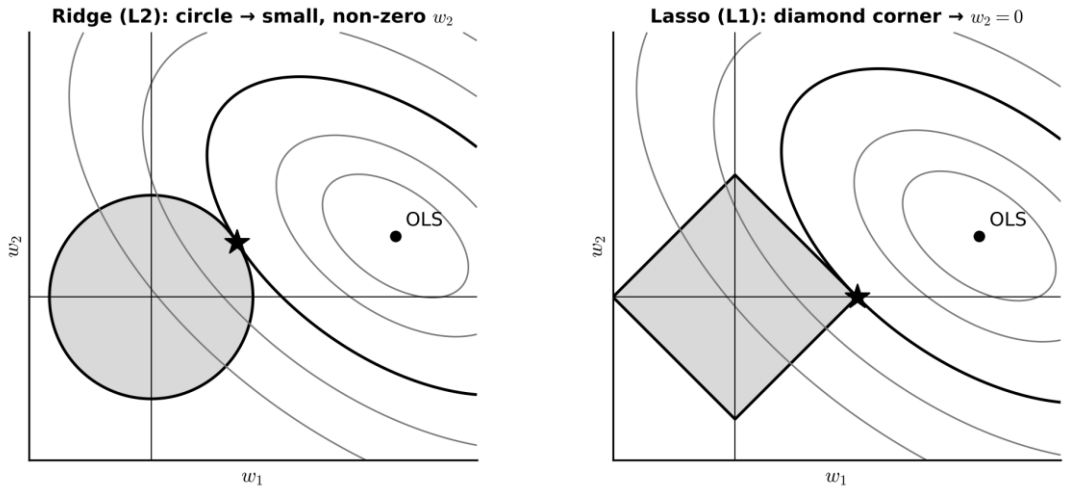


Figure: constrained solutions for the L2 disc and the L1 diamond.

Explanation

Why does the L1 penalty produce exact zeros while L2 doesn't? The geometry explains it.

An equivalent way to write regularisation is as a constrained problem: minimise the squared error subject to a budget on the size of the weights. For ridge, the allowed region is a disc, the L2 ball; for lasso, it's a diamond, the L1 ball, whose corners lie on the axes.

The ellipses are contour lines of the squared error, centred on the unconstrained least-squares solution, labelled OLS. The constrained solution, the star, is the point where the smallest error contour first touches the allowed region.

For the round disc, the first contact point is generally somewhere on the smooth boundary, so both weights are non-zero, here, about 0.84 and 0.54. For the diamond, the contours very often touch first at a corner, and at a corner one of the weights is exactly zero. Here the lasso solution is $w_1 = 1.2$, $w_2 = 0$. In higher dimensions the L1 ball has many corners and edges where several weights are zero at once, which is why lasso produces sparse solutions. The next animation shows this happening as the budget shrinks.

Video: shrinking the budget

Slide 57

Key points

- Large budget: both equal the OLS solution
- Budget shrinks ⇔ λ (α) grows
- Ridge: both weights shrink smoothly, never 0
- Lasso: w_2 snaps to exactly 0 at the corner, then w_1 shrinks
- Final: ridge ≈ (0.20, 0.15) · lasso = (0.25, 0.00)

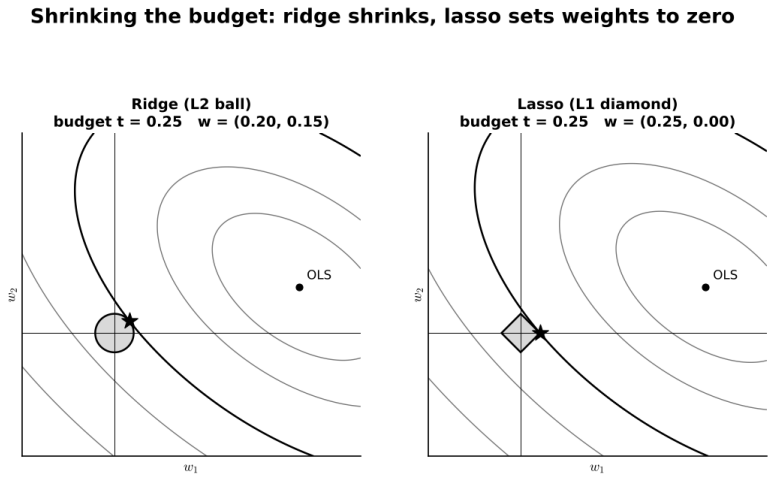


Figure: final frame of the shrinking-budget animation.

Explanation

Click to play. Both panels start with a large budget that contains the unconstrained least-squares solution, so the constraint isn't active and both methods give the OLS weights. Then the budget shrinks, equivalent to increasing λ or alpha.

On the left, the ridge disc shrinks and the solution slides smoothly along its boundary towards the origin: both weights get smaller together, but neither reaches zero.

On the right, watch carefully. Once the diamond's corner on the w_1 axis becomes the contact point, w_2 snaps to exactly 0.00 and stays there, while w_1 continues to shrink. With the final small budget, ridge gives roughly (0.20, 0.15), two small non-zero weights, while lasso gives (0.25, 0.00), one weight, and one feature eliminated.

This is precisely what you'll see in the coefficient paths on real data on the next slide: smooth shrinkage for ridge, coefficients dropping to zero one after another for lasso.

Coefficient paths on real data

Slide 58

Key points

- Diabetes data: 442 patients, 10 features (age, sex, bmi, bp, s1–s6), target = disease progression
- Ridge: smooth shrinkage; collinear coefficients converge quickly
- Lasso: coefficients drop to exactly 0 one by one
- Last survivors: bmi and s5 → most predictive
- Correlated features: lasso picks somewhat arbitrarily

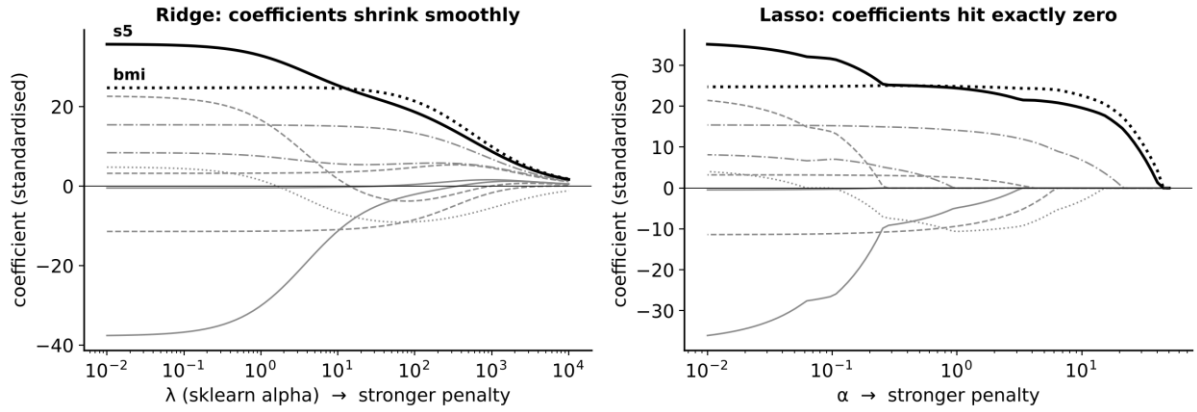


Figure: ridge and lasso coefficient paths on the diabetes data (bold: bmi, s5).

Explanation

Now real data: scikit-learn's diabetes dataset. For 442 patients there are 10 baseline measurements, age, sex, body-mass index, blood pressure and six blood serum measurements, s1 to s6, and the target is a quantitative measure of disease progression one year later. It's the dataset used in the original paper on least-angle regression and a classic lasso example.

The plots show how each of the 10 standardised coefficients changes as the penalty grows from left to right. Two important features are drawn in bold: body-mass index and s5, a serum measurement.

For ridge, on the left, all coefficients shrink smoothly towards zero as λ grows, reaching it only in the limit. Some coefficients that are large and of opposite sign at small λ, a sign of collinearity between the serum measurements, come together quickly.

For lasso, on the right, coefficients hit exactly zero one after another. The last two to survive are bmi and s5: lasso is telling us that these two carry most of the predictive information. Reading a lasso path is a quick way to rank features, but interpret it with care when features are correlated, because lasso tends to pick one of a correlated group somewhat arbitrarily.

Ridge and lasso on the diabetes test set

Slide 59

Key points

- 80/20 split (random_state = 42): 353 train / 89 test
- Baseline MSE 5362 · OLS 2900 (R^2 0.45, 10 features)
- Ridge $\alpha = 0.1$: 2857 · Lasso $\alpha = 0.1$: 2798 (R^2 0.47, 7 features)
- Lasso $\alpha = 1$: 3404, 3 features (bmi, bp, s5) · $\alpha = 2$: 1 feature (bmi)
- Small differences, one split → cross-validation in Week 5

Explanation

Let's measure the effect on unseen patients. We split the diabetes data 80/20 with random_state 42, train on 353 patients and test on 89. The chart shows the test mean squared error.

The mean baseline has a test MSE of 5362. Ordinary least squares, with all 10 features, reduces it to 2900, an R-squared of 0.45, useful, but disease progression is hard to predict. Ridge with alpha 0.1 improves slightly to 2857. Lasso with alpha 0.1 does best here: 2798, R-squared 0.47, while using only 7 of the 10 features. With alpha 1, lasso keeps just three features, bmi, blood pressure and s5, and its error rises to 3404: simpler, more interpretable, a little less accurate. Push alpha to 2 and only bmi remains, with an error close to the baseline.

The lesson: a modest penalty can improve generalisation and simplify the model at the same time; too strong a penalty underfits. The differences here are small and come from a single split of 89 patients, so we should not over-interpret them, in Week 5 you'll learn cross-validation to choose alpha and estimate errors more reliably.

Ridge, lasso or elastic net?

Slide 60

Key points

- Ridge: smooth shrinkage, keeps all features, closed form, stable with collinearity
- Lasso: sparse, feature selection, no closed form, unstable among correlated features
- Elastic net: both penalties, mixing ϱ (l1_ratio)
- Standardise → try both → choose penalty by CV → prefer simpler if similar

Equations

$$\frac{1}{2n}\|X\mathbf{w}-\mathbf{y}\|_2^2+\alpha\rho\,\|\mathbf{w}\|_1+\frac{\alpha(1-\rho)}{2}\|\mathbf{w}\|_2^2$$

Explanation

How do you choose between them? The table summarises the differences.

Ridge shrinks all coefficients smoothly and keeps every feature. It has a closed form, handles collinear features gracefully by spreading weight across correlated features, and is a safe default when you believe many features each contribute a little.

Lasso produces sparse models and performs feature selection, which helps interpretability and is useful when you expect only a few features to matter. But it has no closed form, and among strongly correlated features it tends to pick one and drop the others somewhat arbitrarily, which can make the selected set unstable.

Elastic net, the formula at the bottom, combines both penalties, with a mixing parameter ϱ, l1_ratio in scikit-learn, between pure ridge and pure lasso. It keeps lasso's sparsity while behaving more stably with groups of correlated features, at the price of a second hyperparameter.

In practice: standardise, try ridge and lasso, choose the penalty by cross-validation, and prefer the simpler model when performance is similar. That's exactly the kind of justified model comparison expected in the Week 6 lab and your projects.

Choosing λ: a practical recipe

Slide 61

Key points

- λ is a hyperparameter: training loss would always choose $\lambda = 0$
- Test set untouched → validation set / cross-validation (Week 5)
- Scaler fitted on training data only
- Log grid of λ (10^{-3} … 10^{3}) → best validation error → refit → test once
- Pipeline(StandardScaler, Ridge) · RidgeCV, LassoCV
- Never tune on the test set

Code

```
model = make_pipeline(StandardScaler(),
                      RidgeCV(alphas=np.logspace(-3, 3, 13)))
model.fit(X_train, y_train)
```

Explanation

λ, or alpha, is a hyperparameter: it isn't learned by minimising the training loss, that would always choose $\lambda = 0$, because the penalty can only increase training error. It must be chosen by how well the model generalises.

The recipe. Split off a test set and don't touch it. Within the training data, hold out a validation set, or better, use cross-validation, which you'll learn in Week 5. Fit the scaler on the training part only. Fit the model for a grid of λ values on a logarithmic scale, say from 10^{-3} to 10^{3}. Pick the λ with the lowest validation error, refit on all the training data, and report the test error once.

Scikit-learn makes this convenient. A Pipeline keeps the scaler and the model together, so scaling is always fitted on the right data, you'll build such pipelines in the Week 5 lab. RidgeCV and LassoCV search a grid of penalties with built-in cross-validation.

The one rule you must never break: never choose λ, or anything else, by looking at the test set. Otherwise your reported test error is optimistic, exactly like training error, and you've lost your honest estimate of generalisation.

Part 7, Lab & next steps

Implement it yourself, then compare with scikit-learn

Finally, the lab and what comes next. In the lab you'll practise NumPy, build a baseline, and implement linear regression with the normal equations and with gradient descent from scratch, then explore ridge and lasso. We finish with common pitfalls, readings, key takeaways and an exit ticket.

Today's lab: from NumPy to regularised regression

Slide 63

Key points

- A: NumPy exercises (shapes, broadcasting, @ vs ×)
- B: penguins exploration + baseline (mean/median, DummyRegressor)
- C: normal equations from scratch, checked against lstsq & sklearn
- D: gradient descent from scratch, loss curves, η, scaling, mini-batch
- E: ridge from scratch; ridge & lasso on diabetes; coefficient paths
- F: reflection · deliverable: notebook that runs top to bottom

Explanation

This afternoon's lab has six parts and combines the remaining Week 1 lab tasks with the Week 2 lab.

Part A: NumPy exercises, arrays, shapes, broadcasting, matrix products and vectorised computation of losses, including the classic X-star-w bug. Part B: dataset exploration and a baseline model on the penguins, the mean and median predictors, computed by hand and checked with DummyRegressor. Part C: linear regression from scratch with the normal equations, validated against np.linalg.lstsq and scikit-learn. Part D: gradient descent from scratch, with loss curves, learning-rate experiments, the effect of not scaling, and mini-batch SGD. Part E: ridge from scratch with its closed form, then ridge and lasso with scikit-learn on the diabetes data, including coefficient paths. Part F: a short written reflection.

Every from-scratch result must be checked against a trusted library. Bring your laptop and open the starter notebook in Colab. The deliverable is the completed notebook, run top to bottom without errors.

Common pitfalls when implementing from scratch

Slide 64

Key points

- X × w instead of X @ w → wrong shape, no error
- Missing ones column → line forced through origin
- η too large → loss grows → nan

- Unscaled features → zig-zag, tiny η
- Scaler fitted before splitting → leakage
- Penalising the intercept
- (n,) vs (n, 1) → silent n×n broadcasting
- Gradient without 1/n → η depends on n

Explanation

Here are the bugs I see most often in from-scratch regression code, with their symptoms and fixes. Keep this table open in the lab.

Using star instead of at for matrix products gives a wrong shape without an error. Forgetting the column of ones forces the line through the origin and tilts it. A learning rate that's too large makes the loss grow and then become nan. Unscaled features make gradient descent zig-zag and need a tiny learning rate. Fitting the scaler on all the data before splitting leaks test information. Penalising the intercept in ridge pulls predictions towards zero. Shape mix-ups between vectors of shape (n,) and column matrices of shape (n, 1) broadcast silently into n-by-n matrices. And forgetting to divide by n in the gradient makes the effective learning rate depend on the dataset size.

The defence against all of them is the same: print shapes, plot the loss curve, compare with a library on the same data, and test on a tiny example where you can compute the answer by hand.

Watch and read before next week

Slide 65

Key points

- ISLP Ch. 3 (linear regression), §6.2 (ridge, lasso)
- Géron Ch. 4 “Training Models”
- Deisenroth, Faisal & Ong, Mathematics for Machine Learning (free): Ch. 2, 5, 9
- Videos: StatQuest gradient descent, ridge, lasso; 3Blue1Brown linear algebra
- Revise derivatives of exp and log for Week 3

Explanation

To consolidate today, here are readings and videos; the links are clickable.

For reading: Chapter 3 of An Introduction to Statistical Learning with Python covers linear regression thoroughly, and Section 6.2 covers ridge and lasso, with labs in Python. Chapter 4 of Géron, “Training Models”, covers exactly today's algorithms, the normal equation, batch, stochastic and mini-batch gradient descent, and ridge, lasso and elastic net, with code. For the mathematics, Mathematics for Machine Learning by Deisenroth, Faisal and Ong is free online: Chapter 2 for linear algebra, Chapter 5 for vector calculus, and Chapter 9 for linear regression.

For videos: StatQuest's Gradient Descent, Ridge Regression and Lasso Regression are clear, short introductions. And 3Blue1Brown's Essence of Linear Algebra series builds geometric intuition for

vectors, matrices and projections that will help for the rest of the course, especially for PCA in Week 10.

Before next week, also revise derivatives of the exponential and logarithm functions; we'll need them for logistic regression and maximum likelihood.

Key takeaways

Slide 66

Key points

- Learning = ERM; empirical vs true risk → test on unseen data
- Baseline: mean minimises squared error; R^2 compares with it
- Least squares: $X^TXw = X^Ty$, a projection
- Gradient descent: η and scaling decide convergence
- Ridge shrinks (L2); lasso zeros (L1)
- Tune λ on validation data, never on test

Explanation

Six points to remember. One: learning is empirical risk minimisation, choose the hypothesis with the smallest average training loss, and the gap between empirical and true risk is why we always test on unseen data. Two: always compute a baseline; under squared loss the best constant is the mean, and R-squared measures improvement over it. Three: linear regression is ERM with lines and squared loss; its exact solution satisfies the normal equations, X-transpose-X w equals X-transpose y, which geometrically is a projection. Four: gradient descent minimises any differentiable loss by stepping against the gradient; the learning rate and feature scaling decide whether it converges quickly, slowly, or not at all. Five: ridge adds an L2 penalty that shrinks weights and stabilises the solution; lasso adds an L1 penalty that sets weights exactly to zero. Six: choose the penalty, like every hyperparameter, on validation data, never on the test set.

If you remember one sentence: choose a hypothesis class and a loss, minimise the average loss on training data, by formula or by gradient descent, and control complexity with regularisation.

Exit ticket (3 minutes)

Slide 67

Key points

- Q1: true risk vs empirical risk, and why it matters
- Q2: GD update for linear regression; what if η is too large?
- Q3: 200 features, few relevant, ridge or lasso? Why?

Explanation

Three questions before the lab, on paper or in the Teams channel; not graded.

One: explain in one sentence the difference between true risk and empirical risk, and why it matters. Two: write the gradient-descent update for linear regression with the MSE objective, and say what happens if the learning rate is too large. Three: you have 200 candidate features and expect only a handful to matter. Would you choose ridge or lasso, and why?

Good answers: one, true risk is the expected loss over the whole data distribution, which we cannot compute; empirical risk is the average loss on our training sample; the chosen model's training risk is optimistic, so we must evaluate on unseen data. Two, w becomes w minus η times two over n times X-transpose times (Xw minus y); too large a learning rate overshoots and the loss diverges. Three, lasso, because its L1 penalty sets most coefficients exactly to zero and selects features, perhaps elastic net if the features are strongly correlated.

Closing remarks

Thank you. Let's take a few minutes for questions on any part of today, the maths, ERM, the derivations, gradient descent or regularisation.

After the break we go to the lab. Open the starter notebook in Colab before you arrive if you can. Remember the rule: every from-scratch result is checked against a library, and every gradient-descent run comes with a loss curve.

Next week, in Week 3, we move from regression to classification: logistic regression, which is ERM with a different loss; maximum likelihood, which explains where that loss comes from, and why least squares was maximum likelihood all along; multiclass classification; and Naive Bayes. You'll derive the gradient of the logistic loss yourself, using exactly the chain rule and gradient descent from today.

Appendix A, Derivations, step by step

A.1 The best constant under squared loss is the mean

1. Empirical risk of the constant c: $R(c) = (1/n)\ \Sigma_i\ (y_i - c)^2$.
2. Derivative: $dR/dc = (1/n)\ \Sigma_i\ 2(y_i - c)(-1) = -(2/n)\ \Sigma_i\ (y_i - c)$.
3. Set to zero: $\Sigma_i\ y_i - n\ c = 0 \Rightarrow c^* = (1/n)\ \Sigma_i\ y_i = \bar{y}$.
4. Second derivative: $d^2R/dc^2 = 2 > 0$, so c^* is the (global) minimum.
5. Minimum value: $R(\bar{y}) = (1/n)\ \Sigma\ (y_i - \bar{y})^2$ = the (population) variance of y; its square root, the RMSE of the baseline, is the standard deviation of y.

A.2 The best constant under absolute loss is the median

$R(c) = (1/n)\ \Sigma\ |y_i - c|$. Where it is differentiable, $dR/dc = (1/n)[\#\{y_i < c\} - \#\{y_i > c\}]$: each point below c pushes the loss up as c increases, each point above pushes it down. The derivative changes sign where as many points lie below as above, the median.

A.3 Gradient of the squared error

1. Expand: $\|Xw - y\|^2 = (Xw - y)^T(Xw - y) = w^TX^TXw - 2w^TX^Ty + y^Ty$ (using $w^TX^Ty = y^TXw$, a scalar).
2. Rule 2 with $A = X^TX$ (symmetric): $\nabla(w^TX^TXw) = 2X^TXw$.
3. Rule 1 with $a = X^Ty$: $\nabla(-2w^TX^Ty) = -2X^Ty$.
4. y^Ty does not depend on w. Sum: $\nabla\|Xw - y\|^2 = 2X^T(Xw - y)$; for the mean, $\nabla J(w) = (2/n)X^T(Xw - y)$.

A.4 The normal equations and the simple-regression formulas

Setting $\nabla J = 0$ gives $X^TXw = X^Ty$. With one feature and $X = [1, x]$, the two equations read: $\Sigma(b + w\ x_i) = \Sigma\ y_i$ and $\Sigma\ x_i(b + w\ x_i) = \Sigma\ x_iy_i$. The first gives $b = \bar{y} - w\ \bar{x}$ (the line passes through the point of means). Substituting into the second and simplifying gives $w = \Sigma(x_i - \bar{x})(y_i - \bar{y}) / \Sigma(x_i - \bar{x})^2 = \text{cov}(x, y)/\text{var}(x)$.

A.5 Ridge closed form and weight decay

$J_{ridge}(w) = \|Xw - y\|^2 + \lambda\|w\|^2$. Gradient: $2X^T(Xw - y) + 2\lambda w$. Setting it to zero: $(X^TX + \lambda I)w = X^Ty$. For $\lambda > 0$, $X^TX + \lambda I$ is positive definite (all eigenvalues $\geq \lambda$), hence invertible. Gradient descent on J_{ridge}: $w \leftarrow w - \eta[2X^T(Xw - y) + 2\lambda w] = (1 - 2\eta\lambda)w - 2\eta X^T(Xw - y)$, weight decay.

A.6 Learning-rate stability on a quadratic

For $f(w) = w^2$, the update is $w \leftarrow w - \eta \cdot 2w = (1 - 2\eta)w$, so $w_k = (1 - 2\eta)^k\ w_0$. It converges iff $|1 - 2\eta| < 1$, i.e. $0 < \eta < 1$; $\eta = 0.5$ reaches the minimum in one step. For the standardised penguin regression the Hessian of J is 2I, giving exactly the same condition, which is why $\eta = 1.02$ diverges

and $\eta = 0.5$ converges in one step in the lab. In general the step must satisfy $\eta < 2/L$, where L is the largest eigenvalue of the Hessian; badly scaled features make L large and force a small η.

Appendix B, Key numbers

Quantity	Value
Penguins used	342 (rows with flipper length and body mass)
Mean / median / std of body mass	4201.8 g / 4050 g / 800.8 g
Least squares (all 342)	intercept −5780.83 g, slope 49.69 g/mm; R^2 0.759; RMSE 393.1 g; MAE 313.0 g
Predictions	200 mm → 4156 g · 220 mm → 5150 g
Standardised fit	intercept 4201.75 (= mean), slope 697.64 g per sd (sd 14.04 mm)
Penguins test (80/20, random_state 42)	baseline MAE 693.2, RMSE 819.6 · linear regression MAE 315.9, RMSE 380.7, R^2 0.782
Diabetes test (80/20, random_state 42)	baseline MSE 5361.5 · OLS 2900.2 (R^2 0.453) · ridge α 0.1: 2856.5 · lasso α 0.1: 2798.2 (7 features) · lasso α 1: 3403.6 (bmi, bp, s5)
Learning-rate threshold (standardised penguins)	converges for $\eta < 1$; $\eta = 0.5$ → one step

Appendix C, Self-check questions

Q1. What is the shape of X^TX when X is 342 × 2, and why is it cheap to invert?

Answer: 2 × 2; it is d × d, independent of n.

Q2. Compute ∇f for $f(w_1, w_2) = 3w_1^2 + w_1w_2$ at (1, 2).

Answer: $(6w_1 + w_2, w_1) = (8, 1)$.

Q3. Define true risk and empirical risk. Which one does ERM minimise, and why is the result optimistic?

Answer: True risk: expected loss over $\mathcal{D}$; empirical: average training loss. ERM minimises the empirical risk; the chosen hypothesis is selected to fit this sample, so its training risk underestimates its true risk.

Q4. Which constant minimises the MAE, and why?

Answer: The median: the derivative of the average absolute loss balances the number of points below and above c.

Q5. Write the normal equations and explain them geometrically.

Answer: $X^TXw = X^Ty$: the residual $y - Xw$ is orthogonal to every column of X, so $X\hat{w}$ is the projection of y onto the column space.

Q6. When does X^TX fail to be invertible? What can you do?

Answer: Collinear features or d > n. Remove redundant features, use lstsq, or use ridge ($X^TX + \lambda I$ is invertible).

Q7. Interpret a slope of 49.7 g/mm for penguins. What should you not conclude?

Answer: Each extra mm of flipper is associated with ≈ 50 g more mass; not a causal effect, and not valid outside the observed range.

Q8. Why does gradient descent zig-zag on unscaled features?

Answer: The loss surface is a long, narrow valley; the gradient points mostly across it, and the steep direction forces a small learning rate.

Q9. What is the difference between batch GD, mini-batch GD and SGD?

Answer: The number of examples used per gradient step: all, a small batch, or one; fewer → cheaper, noisier steps.

Q10. Write the ridge update in gradient descent and explain "weight decay".

Answer: $w \leftarrow (1 - 2\eta\lambda)w - 2\eta X^T(Xw - y)$: every step multiplies the weights by a factor below 1 before the data step.

Q11. Why does lasso produce exact zeros but ridge does not?

Answer: The L1 ball has corners on the axes; error contours typically touch it at a corner, where some weights are exactly 0. The L2 ball is smooth.

Q12. Why must features be standardised before ridge or lasso?

Answer: The penalty treats all weights equally; without standardisation the penalty depends on arbitrary units.

Q13. How should λ be chosen?

Answer: On validation data or by cross-validation, with the scaler fitted on training data only, never on the test set.

Appendix D, References and resources

- James, G., Witten, D., Hastie, T., Tibshirani, R., & Taylor, J. (2023). *An Introduction to Statistical Learning with Applications in Python*. Springer., Ch. 3, §6.2.
- Géron, A. (2022). *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow* (3rd ed.). O'Reilly., Ch. 4 Training Models.
- Deisenroth, M. P., Faisal, A. A., & Ong, C. S. (2020). *Mathematics for Machine Learning*. Cambridge University Press. Free at mml-book.github.io, Ch. 2, 5, 9.

- Tibshirani, R. (1996). Regression shrinkage and selection via the lasso. *Journal of the Royal Statistical Society B*, 58(1), 267–288.
- Hoerl, A. E., & Kennard, R. W. (1970). Ridge regression: biased estimation for nonorthogonal problems. *Technometrics*, 12(1), 55–67.
- Efron, B., Hastie, T., Johnstone, I., & Tibshirani, R. (2004). Least angle regression. *Annals of Statistics*, 32(2), 407–499. (diabetes data)
- Gorman, K. B., Williams, T. D., & Fraser, W. R. (2014). Ecological sexual dimorphism and environmental variability within a community of Antarctic penguins (genus *Pygoscelis*). *PLoS ONE*, 9(3), e90081.
- StatQuest, Gradient Descent: https://youtu.be/sDv4f4s2SB8 · Ridge: https://youtu.be/Q81RR3yKn30 · Lasso: https://youtu.be/NGf0voTMlcs
- 3Blue1Brown, Essence of Linear Algebra: https://www.3blue1brown.com/topics/linear-algebra